Chair of Network Architectures and Services
School of Computation, Information, and Technology
Technical University of Munich



# TECHNICAL UNIVERSITY OF MUNICH

## SCHOOL OF COMPUTATION, INFORMATION, AND TECHNOLOGY

## INFORMATICS

MASTER'S THESIS IN INFORMATICS

# QUIC Improvements Using XDP

Dominik Ritzenhoff

Technical University of Munich
School of Computation, Information, and Technology
Informatics

Master's Thesis in Informatics

# QUIC Improvements Using XDP

# Verbesserung der Performanz von QUIC mithilfe von XDP

| | |
|---|---|
| Author: | Dominik Ritzenhoff |
| Supervisor: | Prof. Dr.-Ing. Georg Carle |
| Advisor: | Johannes Späth |
| | Marcel Kempf |
| Date: | July 21, 2025 |

I confirm that this Master's Thesis is my own work and I have documented all sources and material used.

Garching, July 21, 2025

Location, Date

Signature

## Abstract

QUIC is a general-purpose transport protocol, first standardized by the Internet Engineering Task Force (IETF) in 2021, which is developed on top of UDP and combines features from mature protocols such as TCP and TLS. QUIC differentiates itself in that it offers secure connections by default, promises fewer round trips before data is transmitted, fixes the head-of-line blocking present with TCP, and is designed to be extensible through its user space implementations. However, previous work has shown packet I/O to be QUIC's largest bottleneck in terms of its maximum achievable goodput in high-bandwidth networks, and many QUIC implementations fail to reach the same goodputs as TCP+TLS in such environments. This highlights a need for further optimization of the UDP+QUIC datapath to match TCP's performance.

In this thesis, we investigate adding *EXpress Data Path (XDP)* to the QUIC datapath to analyze its impact on latency and reducing packet I/O. We use the MsQuic implementation from Microsoft as it offers support for XDP. We show that while XDP is not able to acheive a higher goodput than MsQuic with the UDP socket optimizations *message batching* and *segmentation offloading*, it is significantly higher than the MsQuic datapath with no optimizations. To analyze latency, we sync the hardware clocks and system time between a packet sniffer and client using Precision Time Protocol (PTP) and analyze the time difference between receiving the packet in userspace within the client and receiving the packet on the NIC in the sniffer. We show that there is no significant difference between the XDP and non-XDP cases. Finally, we create a taxonomy in which we evaluate various QUIC optimizations addressing the packet I/O limitation, organizing each based on implementation complexity, performance improvement potential, unwanted side effects, privilege level usage, and deployment.

I would like to express my sincere gratitude to Prof. Dr.-Ing. Georg Carle, as well as my advisors Johannes Späth and Marcel Kempf, for the opportunity to work on this fascinating topic. I especially thank my advisors for their dedicated supervision, generous support, and insightful guidance throughout the course of this work. And to my friends, thanks for all the Mensa chats, the delicious food and drink we shared, and the hearty laughs that brightened our days.

Finally, I owe a special thanks to my parents, Katti and Michael, for their unwavering support, encouragement, and patience, which have been invaluable to me throughout my studies. To my siblings, Lea and Jan, I love you both and am deeply grateful to have had the privilege of growing up with you. I'm excited to see what the next chapter holds for all of us. To Birgit (a.k.a. ama), thank you for being the best grandmother I could have ever asked for, and for giving me a home away from home these last two years.

# Contents


**1 Introduction** **1**

1.1 Research Questions . . . . . 2

1.2 Outline . . . . . 2

**2 Background** **5**

2.1 A TCP and QUIC Comparison . . . . . 5

2.1.1 Head-of-Line Blocking . . . . . 5

2.1.2 Iteration Velocity . . . . . 6

2.1.3 Transport and Cryptographic Handshakes . . . . . 8

2.1.4 Security, Privacy, and Ossification . . . . . 8

2.1.5 Connection Migration . . . . . 9

2.1.6 Packet Ambiguity . . . . . 9

2.1.7 QUIC's Limitations . . . . . 9

2.2 eBPF . . . . . 11

2.2.1 Instruction Set . . . . . 11

2.2.2 Helper Functions . . . . . 12

2.2.3 Maps . . . . . 12

2.2.4 Limitations . . . . . 14

2.3 XDP . . . . . 14

2.3.1 Metadata . . . . . 15

2.3.2 XDP Actions . . . . . 17

2.3.3 AF_XDP . . . . . 17

2.4 Precision Time Protocol . . . . . 22

2.4.1 PTP Profiles . . . . . 23

2.4.2 PTP Tools . . . . . 23

2.4.3 Time Types . . . . . 24

2.4.4 NIC Hardware Clocks . . . . . 25

2.4.5 Clock Types . . . . . 26

2.4.6 Best Master Clock Algorithm . . . 28
2.4.7 Synchronization . . . 29
2.5 MsQuic . . . 31
2.5.1 Receiving Data . . . 33
2.5.2 Logging . . . 35
2.6 pos Measurement Framework . . . 36

**3 Implementation** **37**
3.1 Testbed Setup . . . 37
3.2 Integrating AF_XDP into the QIR . . . 38
3.2.1 XDP Program Flow . . . 38
3.2.2 Fixing the Release Build Failure on Debian Bookworm . . . 38
3.2.3 Enabling XDP Within the Client . . . 40
3.2.4 Optimizing the XDP Datapath . . . 41
3.2.5 MsQuic Release-Mode Goodput Test Failures . . . 41
3.2.6 Variation of Batch Sizes . . . 43
3.2.7 Driver Mode, Zero-Copy Driver Mode, and SKB Mode . . . 43
3.3 Integrating Precision Time Protocol into the QIR . . . 43
3.3.1 Measuring the Network Stack Traversal Time . . . 45

**4 Evaluation** **49**
4.1 XDP Modes . . . 50
4.1.1 AF_XDP on the Server . . . 50
4.1.2 AF_XDP on the Client . . . 54
4.1.3 RX Batch Sizes . . . 54
4.2 NIC to Userspace Packet Traversal time with PTP . . . 56

**5 Taxonomy over QUIC Datapaths** **59**
5.1 Performance Improvement Potential . . . 59
5.2 Implementation Complexity . . . 60
5.3 Unwanted Side Effects . . . 61
5.4 Privilege Level Usage and Deployment . . . 61

**6 Related Work** **63**

**7 Conclusions and Future Work** **67**
7.0.1 Performance Improvements . . . 68

**8 Appendix** **71**

**Bibliography** **75**

# List of Figures


2.1 HTTP/2 and HTTP/3 Network Stacks. . . . . 6
2.2 Head-of-Line Blocking. . . . . 7
2.3 Handshake Comparison Between TCP and QUIC. . . . . 8
2.4 Maximum Memory Bandwidth with TCP. . . . . 10
2.5 Maximum Memory Bandwidth with QUIC. . . . . 11
2.6 BPF Map. . . . . 13
2.7 XDP Hook Points. . . . . 15
2.8 UMEM After Initialization. . . . . 19
2.9 UMEM Chunks Owned by Fill Queue. . . . . 19
2.10 UMEM Chunks Owned by Kernel. . . . . 20
2.11 UMEM after Kernel Adds Descriptors to RX Queue. . . . . 21
2.12 UMEM after MsQuic Adds Descriptor to TX Queue. . . . . 21
2.13 A PTP Topology showing phc2sys usage for system time and PHC discipline. . . . . 24
2.14 PTP Timestamping Options. . . . . 26
2.15 Example Precision Time Protocol (PTP) Network with Pruning. . . . . 27
2.16 Transparent Clock Behavior in a P2P PTP Setup. . . . . 28
2.17 PTP Clock Frequency Difference Calculation . . . . . 31
2.18 PTP Delay and Offset Calculations . . . . . 32
2.19 Components of the Default Packet Datapath. . . . . 33
2.20 Packet Reception with `recvfrom(2)`. . . . . 34
2.21 Packet Reception with `recvmmsg(2)`. . . . . 34
2.22 Packet Reception with Generic Receive Offload (GRO). . . . . 35
2.23 Packet Reception with GRO and `recvmmsg(2)`. . . . . 35
2.24 Measurement Topology of the QUIC Interop Runner (QIR). . . . . 36

3.1 XDP Program Flow. . . . . 39
3.2 XDP Datapath Initialization before Optimization. . . . . 42
3.3 XDP Datapath Initialization after Optimization. . . . . 42

3.4 Packet Reception with AF_XDP SKB Mode. . . . . 44
3.5 Packet Reception with AF_XDP Driver Mode and Zero-copy Disabled. . . 44
3.6 Packet Reception with AF_XDP Driver Mode and Zero-copy Enabled. . . 44
3.7 Encryption within the QUIC Header. . . . . 46

4.1 Goodput vs XDP Modes on AF_XDP Server with mlx5_core Driver. . . . 51
4.2 Goodput vs XDP Modes on AF_XDP Server with ixgbe Driver. . . . . 51
4.3 User and System CPU Usage vs XDP Modes on AF_XDP Server with ixgbe Driver. . . . . 52
4.4 Flamegraph of XDP_DRV with Zero-Copy Enabled. . . . . 53
4.5 Flamegraph of XDP_SKB. . . . . 53
4.6 Goodput vs XDP Modes on AF_XDP Client with ixgbe Driver. . . . . 55
4.7 Goodput vs XDP Modes on AF_XDP Client with mlx5_core Driver. . . . 55
4.8 User and System CPU Usage vs XDP Modes on AF_XDP Client with ixgbe Driver. . . . . 56
4.9 Goodput vs RX Batch Size on AF_XDP Client with ixgbe Driver. . . . . 57
4.10 Userspace-NIC Timestamp Differences as Boxplots. . . . . 58
4.11 Userspace-NIC Timestamp Differences as a CDF Plot. . . . . 58

# List of Tables


2.1 eBPF Register Overview . . . . . 12
2.2 Generic eBPF Map Types . . . . . 13
2.3 Non-Generic (Specialized) eBPF Map Types . . . . . 14
2.4 XDP Program Return Actions . . . . . 17
2.5 Key Attributes for PTP Best Master Clock Algorithm (BMCA) . . . . . 29

5.1 Taxonomy of QUIC Datapaths. . . . . 59

# Chapter 1

# Introduction

Building upon foundational elements from the Transmission Control Protocol (TCP) [1] and Transport Layer Security (TLS) [2], QUIC [3] is a general-purpose transport protocol developed atop the User Datagram Protocol (UDP), and offers several advantages over the traditional TCP+TLS stack. Its key differentiators include integrated security via TLS, reduced round trips for data transmission by consolidating connection establishment and secure channel negotiation, mitigation of TCP's head-of-line blocking, and enhanced extensibility through userspace implementations. These capabilities position QUIC as an ideal foundation for protocols like Hypertext Transfer Protocol 3 (HTTP/3) [4], which already accounts for 20.5% of global HTTP traffic [5]. This increasing adoption also underscores the critical need for optimizing QUIC's performance. Seen in the work from Jaeger et al. [6], packet I/O represents the most significant bottleneck to QUIC's maximum achievable goodput, and it highlights the unique challenges QUIC faces being a userspace protocol due to the inherent costs of context switches and there being fewer mature optimizations and offloads than TCP.

Fortunately, EXpress Data Path (XDP) offers one way to reduce packet I/O within the Linux kernel. Introduced in 2016 [7], XDP functions as a hook in Linux's network receive (RX) path, enabling packet handling at the earliest possible stage, prior to any other kernel processing. With AF_XDP, a specific socket type, raw Layer 2 frames can be directly passed from the XDP hook point to userspace QUIC implementations. While requiring some additional header processing in userspace, AF_XDP allows packets to bypass the Linux network stack entirely.

Beyond maximizing goodput, many applications are highly latency-sensitive. Consequently, our research also examines the time difference for a packet to traverse from the

Network Interface Card (NIC) into userspace when utilizing XDP and MsQuic's default datapath. We use PTP to make these measurements possible.

Finally, it is important to note that XDP represents just one approach to mitigating the packet I/O overhead. Alternatives include Data Plane Development Kit (DPDK) [8], a software framework providing libraries and NIC polling-mode drivers to offload all packet processing from kernel space to userspace. Another promising alternative is in-kernel QUIC [9], which relocates QUIC into the kernel. In sum, each technology offers different benefits and drawbacks, which highlights the need to create a taxonomy in which we evaluate these QUIC optimizations, organizing each based on implementation complexity, performance improvement potential, unwanted side effects, privilege level usage, and deployment.

## 1.1 Research Questions

In this thesis, we answer the following research questions:

**To what extent can we increase QUIC performance by using XDP in a trivial way? Is there optimization potential?** We interpret performance as both maximizing goodput and minimizing latency. MsQuic is used for this research as it sets the bar high with its optimized default datapath, which enables it to have one of the highest goodputs out of any QUIC implementation [10]. Moreover, it is one of the few implementations to offer rudimentary support for XDP. A thorough understanding of XDP's performance may therefore serve as a guide for other QUIC implementations that are looking to become more optimized in both goodput and latency.

**Can a taxonomy be created for QUIC optimizations, categorizing them by implementation complexity, performance improvement potential, unwanted side effects, privilege level usage, and deployment?** As previously alluded to, there exist multiple methods to reduce packet I/O within QUIC. Each method presents its own set of tradeoffs with respect to the concepts introduced in the research question. Elaborating on each concept will give deeper insight when it is appropriate to use each technology.

## 1.2 Outline

In Chapter 2, we provide additional context on the QUIC protocol, XDP, AF_XDP, the QIR used to gather goodput and latency data, MsQuic and its default datapath optimizations, and PTP. Chapter 3 describes the adaptations made to MsQuic and the QIR in order to enable our measurements. We also describe what we measured and

our procedure for doing so. Chapter 4 investigates the results of our measurements and offers explanations for them. Finally, Chapter 5 creates a taxonomy over alternative QUIC datapaths, Chapter 6 investigates other relevant work, and Chapter 7 summarizes our findings and provides an outline for future work.

# Chapter 2

# Background

This chapter lays the groundwork for our investigations by covering several key areas. We first provide foundational information on the QUIC protocol by comparing it to TCP. Subsequently, we cover extended Berkeley Packet Filter (eBPF), XDP, and AF_XDP, followed by an introduction to PTP, and finally, an overview of the QIR framework.

## 2.1 A TCP and QUIC Comparison

QUIC [3] is a next-generation transport protocol developed to address the inherent limitations of protocols like TCP and TLS. It offers several key advantages: a combined transport and cryptographic handshake for faster connection establishment, native stream multiplexing to prevent head-of-line blocking and improved goodput resilience to packet loss, robust connection migration for seamless transitions between networks, and easier deployment.

Google, the original developer of QUIC, was already serving 50% of all requests to Google services from Chrome browsers over QUIC by 2015 [11]. Today, HTTP/3, which uses QUIC, is enabled by default on Chrome browsers [12]. These statistics underscore the protocol's clear benefits and its widespread adoption across the Internet. Figure 2.1 illustrates the respective technology stacks of HTTP/3 and its predecessor, HTTP/2. The following paragraphs dive deeper into implications of such changes.

### 2.1.1 Head-of-Line Blocking

QUIC was specifically designed to overcome several of TCP's inherent shortcomings, most notably Head of Line (HoL) blocking. Similar to QUIC, TCP's design mandates

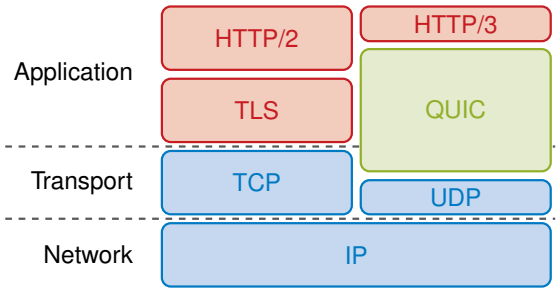



Figure 2.1: HTTP/2 and HTTP/3 Network Stacks.

the in-order delivery of packets within a single connection. However, the problem in TCP arises when even a single packet is delayed or dropped en-route. For higher-level protocols like HTTP/2 that build off TCP, this means all subsequent packets are forced to wait for the retransmission of the lost packet, irrespective of the application-level stream they belong to. Figure 2.2 illustrates this phenomenon. Consequently, this property makes HTTP/2 highly vulnerable to performance degradation from packet loss, especially when utilizing multiple streams. With QUIC, the concept of streams is native to the protocol itself, which lets HTTP/3 effectively multiplex without any risk of HoL blocking.

### 2.1.2 Iteration Velocity

Integrating TCP's codebase directly within the operating system kernel, while beneficial for avoiding costly packet copying to userspace, introduces substantial drawbacks. This approach leads to a comparatively slow adoption rate for new features or bug fixes, and significantly raises the stakes for every code change. For any new TCP enhancement to achieve widespread use, it requires all clients to upgrade their kernel versions to the one where the feature was introduced. This tight coupling creates brittleness for applications dependent on particular TCP features, as they may not function correctly if paired with an outdated kernel. This contrasts sharply with protocols like QUIC, which operates in userspace and is integrated directly into the application itself. Consequently, updating QUIC is as simple as bundling the newest version of the QUIC library with the application code, thereby eliminating this brittleness entirely. Furthermore, userspace offers a more forgiving environment for experimentation. If a new algorithm or feature contains a bug, it is far less likely to crash the entire operating system than with TCP, making development and testing safer and quicker.

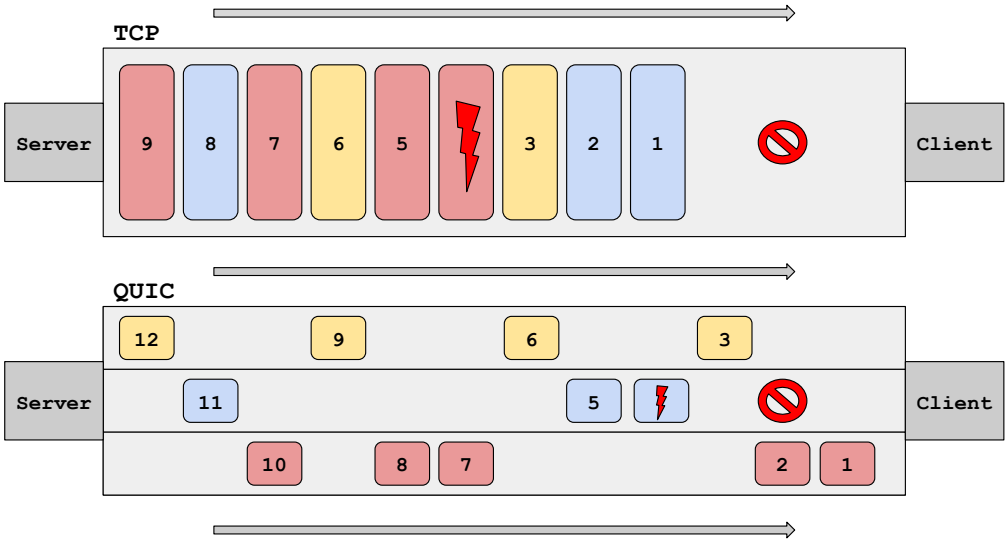



Figure 2.2: TCP and QUIC Streams with Packet Loss. The TCP client will receive all packets from the three streams but packet 4, which went missing. Head-of-Line blocking ensures that the HTTP/2 client only receives packets 1, 2, and 3. Packet 4 must be resent for the HTTP/2 client to receive any subsequent packets. The QUIC client will receive all packets but packet 4. The HTTP/3 client will receive all packets but 4, 5, and 11, which are part of the one stream that is blocked.

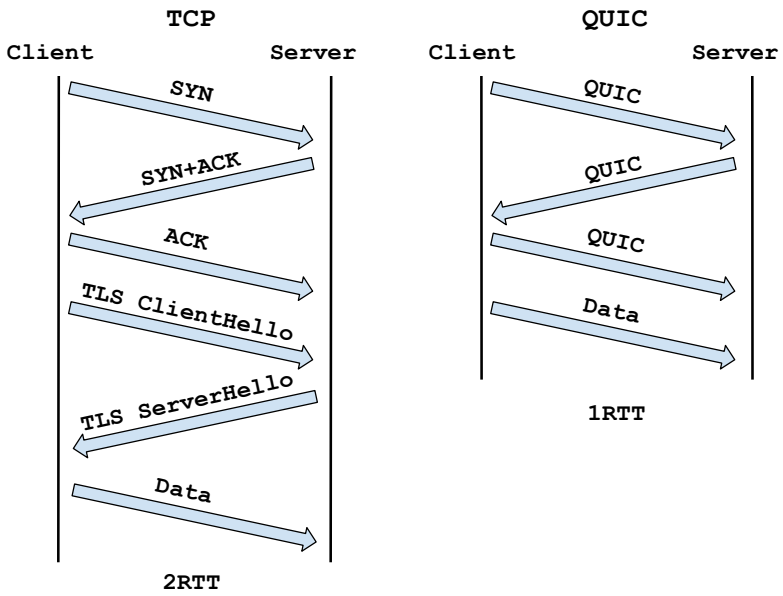



Figure 2.3: Handshake Comparison Between TCP and QUIC.

### 2.1.3 Transport and Cryptographic Handshakes

With TCP+TLS, a round trip is needed to establish the TCP connection, and another round trip is then required to make it secure. QUIC, however, integrates TLS directly into its protocol, combining these two steps into a single handshake, as illustrated in Figure 2.3. This integration typically leads to a reduced time-to-first-byte. Additionally, in the special case that QUIC has previously communicated with a server, it can leverage cached cryptographic parameters and connection settings to transmit application data during the very first round-trip [3]. This is known as 0 Round Trip Time (0-RTT).

### 2.1.4 Security, Privacy, and Ossification

Another downside to TLS and TCP is that they leave header information exposed to middleboxes. In fact, none of TCP's header components is encrypted. This exposure contributes to a larger attack surface area for malicious actors but also contributes to network ossification. Middleboxes can enforce outdated TCP behaviors and hinder protocol evolution by overwriting visible values or by expecting specific field contents and dropping packets that don't adhear to them. In contrast, QUIC encrypts a substantial portion of its header fields, including the packet number, key phase, and reserved bits, making it far less susceptible to such issues. Even fields not fully encrypted, like the Connection ID, are integrity-protected. This design allows for immediate detection of

packet tampering and severely impedes a Man-in-the-Middle (MITM) attacker from altering values. While TCP and TLS technically also provide integrity protection by means of the checksum field, the checksum is a simple 1's complement sum, which is feasible for an attacker to recompute.

To further combat ossification, QUIC also employs a technique known as greasing. One instance of such usage is with the 'QUIC Bit', which is the second most significant bit of the first byte in every QUIC packet. While this bit is typically fixed and is used to aid in protocol identification, QUIC endpoints may negotiate using the `grease_quic_bit` transport parameter to send an arbitrary value instead [13]. This consequently encourages middleboxes to tolerate unknown or future extensions rather than relying on a static value.

### 2.1.5 Connection Migration

Unlike TCP, which binds connections to the source and destination IP addresses and ports, QUIC maintains connections with Connection ID (CID)s. The advantage is that it allows QUIC to migrate across underlying network changes, such as from Wi-Fi to cellular data, without needing to re-establish the entire connection, as is the case for TCP. Even when the source IP changes, the client may continue using the same CID. This is especially useful for mobile environments given the almost ubiquitous support of both Wi-Fi and cellular.

### 2.1.6 Packet Ambiguity

When a packet goes missing in TCP, a retransmission from the server to the client carries the same sequence number and data as the original. This introduces ambiguity, making it difficult to discern if an incoming packet is a delayed original or a new copy. QUIC resolves this ambiguity through the use of monotonically growing packet numbers. If a packet is lost, a new packet will be generated with a unique, incremented packet number, potentially containing retransmitted payload. An additional benefit of this approach is the enhanced visibility of packet loss, which in turn enables more fine-grained tuning of QUIC's congestion control algorithms.

### 2.1.7 QUIC's Limitations

Despite its many benefits, QUIC faces existing drawbacks that hinder its performance, especially within the context of high-bandwidth networks. As demonstrated by Jaeger et al. [6], TCP and TLS achieved a goodput of 8010 Mbit/s on a 10 Gbit/s link, whereas the top-performing QUIC implementation measured only 3004 Mbit/s. As previously

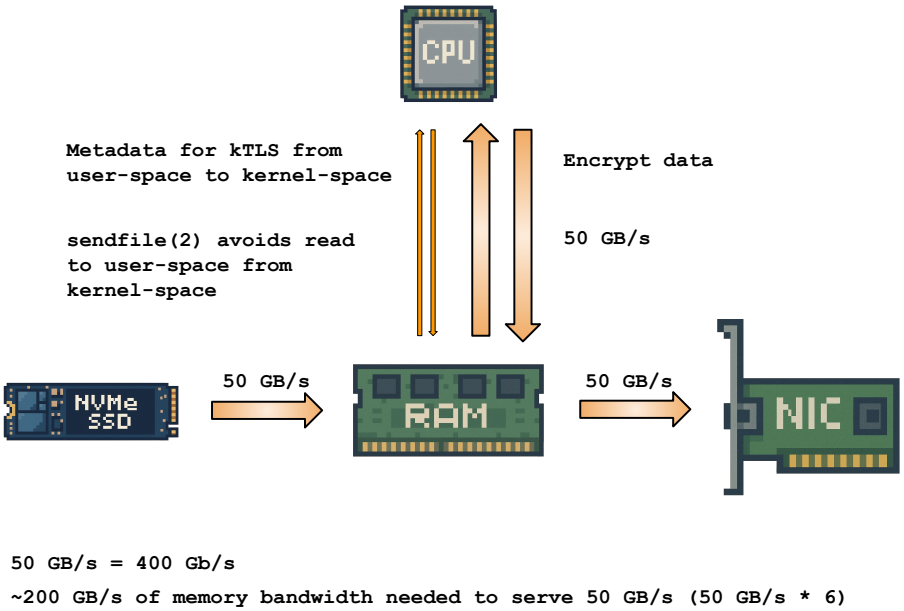



Figure 2.4: Maximum Memory Bandwidth with TCP.

discussed in Chapter 1, this performance disparity is largely attributable to inefficient packet I/O, which can be optimized. However, other significant challenges exist.

Consider a scenario where a server, acting like a streaming service, sends static data stored in non-volatile memory to a client. A TCP-based solution leveraging Kernel Transport Layer Security (kTLS) and the `sendfile(2)` system call (as illustrated in Figure 2.4) can efficiently transfer data with only a single copy into kernel memory from storage. In contrast, because QUIC operates as a userspace application, this process involves an additional memory copy: first from storage into kernel memory, and then from kernel memory into the application's userspace memory. As depicted in Figure 2.5, this incurs not only increased CPU cycles for the copying but also consumes significant memory bandwidth. While introducing a new API could potentially eliminate this extra copy, such a change would carry serious security implications by blurring the critical distinction between kernel and userspace capabilities.

This all goes to say that QUIC itself is not a silver bullet, and there exist many areas for improvement.

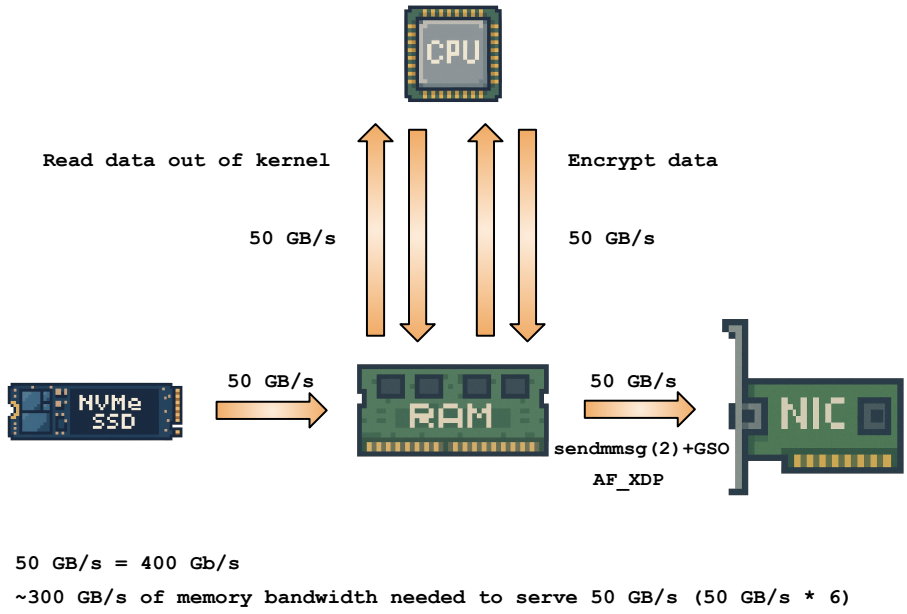



Figure 2.5: Maximum Memory Bandwidth with QUIC.

## 2.2 eBPF

eBPF, also simply known as Berkeley Packet Filter (BPF), is a Linux-native in-kernel bytecode language and virtual machine that allows for programmable, event-driven execution across many of the kernel's subsystems. It represents an evolution from classic Berkeley Packet Filter (cBPF), which also offers an in-kernel bytecode language and virtual machine but with fewer capabilities. The work flow for creating an eBPF program typically entails writing the program in a subset of C, compiling it using a tool like LLVM, writing the userspace loader program, and finally loading and attaching the program. Once a specific event occurs, the eBPF program will subsequently be invoked. Finally, because it is a Linux-native technology, BPF comes packaged with every kernel version since 3.18 [14].

### 2.2.1 Instruction Set

BPF is a general purpose Reduced Instruction Set Computer (RISC) architecture which can be used as a compilation target for compiler back ends. Typically written in a subset of C that does not allow features such as arbitrary loops, unbounded recursion, and arbitrary memory accesses, BPF bytecode is loaded by the kernel at runtime and is later Just in Time (JIT) compiled into native opcodes for better performance. This

approach allows the kernel to become programmable without having to cross the user-kernel space boundary. This is incredibly advantageous because it offers a flexible new way of gaining visibility into the Linux kernel as well as boosting datapath performance by enabling data handling at the earliest possible stages. Moreover, BPF programs can be atomically updated at runtime without requiring a kernel restart, allowing for dynamic changes throughout a program's lifecycle.

BPF consists of eleven 64 bit registers with 32 bit subregisters. It also contains a 512 byte stack space and a program counter. Each of the eleven registers are named `r0 - r10`, and the default operating mode is 64 bits. The 32 bit subregisters are only accessible through ALU operations. Table 2.1 shows the BPF calling convention.

| Register | Type | Description |
|---|---|---|
| r0 | R/W | Exit value for BPF program. |
| r1-r5 | R/W | Scratch registers; hold arguments to the kernel helper function. |
| r6-r9 | R/W | Callee-saved scratch registers. |
| r10 | Read-Only | Frame pointer for accessing the BPF stack. |

Table 2.1: eBPF Register Overview

Because of the simple nature of BPF's calling convention, it can map directly to `x86_64` and `arm64`, which greatly benefits the JIT compiler given that it needs only to issue a single call instruction when placing function arguments.

### 2.2.2 Helper Functions

Depending on the hook point, a given eBPF program will have access to a number of helper functions, which provide assistance in retrieving and pushing data to the kernel. Using the registers defined in Table 2.1, a typical helper function signature will take a form similar to `u64 fn(u64 r1, u64 r2, u64 r3, u64 r4, u64 r5)`. To ensure a stable and architecture-agnostic interface, the kernel then abstracts helper functions using the macros `BPF_CALL_0` to `BPF_CALL_5`. These macros precisely define how arguments are passed from the eBPF virtual machine's registers to the kernel helper function, allowing the JIT compiler to issue direct call instructions across various CPU architectures.

### 2.2.3 Maps

eBPF maps represent the way information is passed between the eBPF program and userspace application. They are key-value stores that live inside the kernel and are the

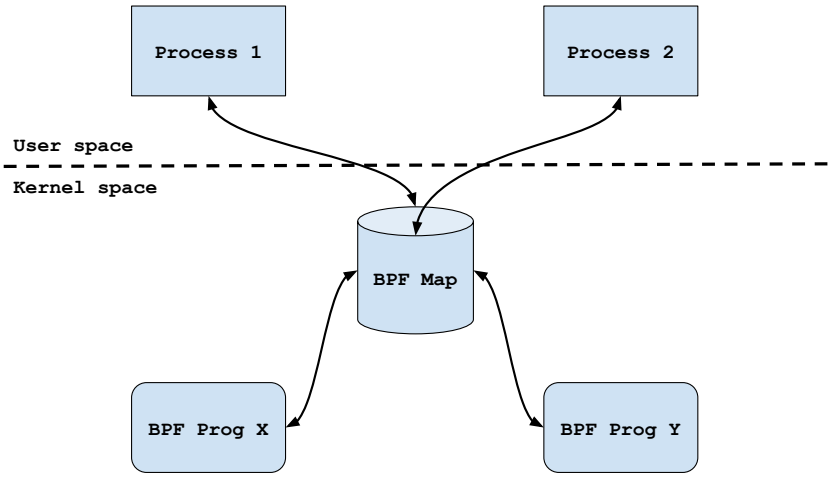



Figure 2.6: BPF Map.

data structure typically used for managing state within eBPF programs. They also allow data to persist across multiple program executions. As seen within Figure 2.6, processes in userspace access maps through file descriptors, and a map may also be shared between multiple processes and BPF programs.

There are also many map types available as seen in Table 2.2 and Table 2.3. While each generic map type is functionally different, all the map types use a common set of BPF helper functions for lookups, updates, and deletions. When these data structures aren't sufficient, non-generic map types may be used to fill the gaps.

| Generic Map Type | Description |
|---|---|
| `BPF_MAP_TYPE_HASH` | Standard hash table. |
| `BPF_MAP_TYPE_PERCPU_HASH` | Per-CPU hash table. |
| `BPF_MAP_TYPE_LRU_HASH` | Hash table with a Least Recently Used (LRU) eviction policy. |
| `BPF_MAP_TYPE_LRU_PERCPU_HASH` | Per-CPU Hash table with a per CPU Least Recently Used (LRU) eviction policy. |
| `BPF_MAP_TYPE_ARRAY` | Standard array, indexed by integer keys. |
| `BPF_MAP_TYPE_PERCPU_ARRAY` | Per-CPU array. |
| `BPF_MAP_TYPE_LPM_TRIE` | Longest Prefix Match Trie, useful for routing and IP address lookups. |

Table 2.2: Generic eBPF Map Types

| Non-Generic Map Type | Description |
|---|---|
| `BPF_MAP_TYPE_XSKMAP` | An array used by XDP programs to redirect network packets to AF_XDP sockets. |
| `BPF_MAP_TYPE_PROG_ARRAY` | An array that holds pointers to other eBPF programs. Used for complex control flow or "tail calls". |
| `BPF_MAP_TYPE_PERF_EVENT_ARRAY` | Used for sending data from eBPF programs to userspace via the `perf` subsystem's ring buffers. |
| `BPF_MAP_TYPE_CGROUP_ARRAY` | Used to manage BPF programs attached to cgroups. |
| `BPF_MAP_TYPE_STACK_TRACE` | Captures kernel or userspace stack traces for profiling and debugging. |
| `BPF_MAP_TYPE_ARRAY_OF_MAPS` `BPF_MAP_TYPE_HASH_OF_MAPS` | Maps that hold pointers to other maps, allowing for atomic replacement of entire maps at runtime. |

Table 2.3: Non-Generic (Specialized) eBPF Map Types

### 2.2.4 Limitations

While powerful, eBPF isn't a silver bullet. Its capabilities remain significantly more constrained than those of a typical userspace application. For example, it doesn't support function calls with six or more arguments, the strict verifier might reject a program if it can't definitively prove termination, and eBPF programs have access to only a very limited set of helper functions and a small stack size. Finally, debugging is basic. It is currently not possible to step through an eBPF program while it runs in the kernel, which consequently makes print debugging the most effective method. For most use cases, userspace applications are still the most viable option.

## 2.3 XDP

XDP is a hook point situated very early in the network receive path, and it allows a loaded BPF program to be invoked for every packet. Seen in Figure 2.7, there are two XDP hook points: `XDP_DRV` and `XDP_SKB`. The former represents the earliest point at which a packet may be processed in software, but it comes with the cost of requiring integration into the network driver. When this is not an option, `XDP_SKB` offers a generic hook point which functions for every setup, but requires additional per-packet processing.

One stipulation to loading an XDP program is that it may only be attached to a single queue on a specific NIC. This is an important consideration as packet steer-

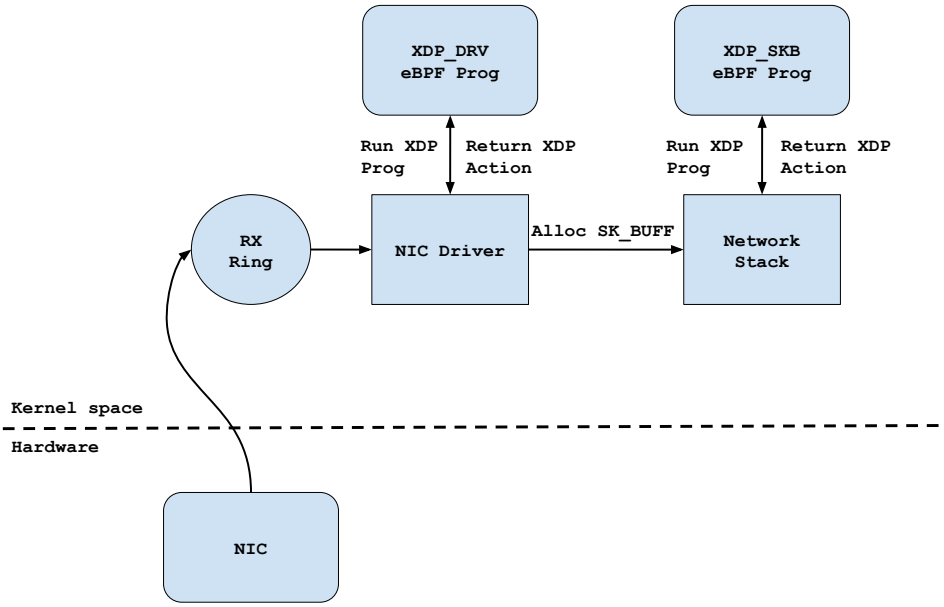



FIGURE 2.7: XDP Hook Points.

ing mechanisms like Receive Side Scaling (RSS) are often used to distribute different flows across the queues of a particular NIC. Therefore, to capture all relevant traffic, it is crucial to either attach the program to every single queue or to combine the queues together using a command like `sudo ethtool -L <interface_name> combined <number_of_queues>`. This directs the NIC to use a specified number of combined receive and transmit queues.

### 2.3.1 METADATA

Depending on whether `XDP_DRV` or `XDP_SKB` is used, access to different sources of information is available, presenting a trade-off between early access to raw data and later access to richer, more context-aware information.

#### XDP__MD

Upon entry of every single `XDP_DRV` program, a struct `xdp_md` is passed as an argument, providing essential information about the packet. Listing 2.1 shows the struct's definition, which contains pointers to the beginning and end of the data as well as the index of the network interface on which the packet was received, the receive queue that handled the packet, and a metadata field where network drivers or XDP programs can

store custom information associated with the packet. As seen in Figure 2.7, an `sk_buff` has not yet been allocated, which is the reason for having less metadata available.

Listing 2.1: The xdp_md struct

```
struct xdp_md {
        __u32 data;
        __u32 data_end;
        __u32 data_meta;
        /* Below access go through struct xdp_rxq_info */
        __u32 ingress_ifindex; /* rxq->dev->ifindex */
        __u32 rx_queue_index;  /* rxq->queue_index  */
        __u32 egress_ifindex;  /* txq->dev->ifindex */
};
```

### SK_BUFF

The `sk_buff` struct, which stands for socket buffer, and whose definition can be seen in Listing 2.2, is the central data structure representing a network packet as it traverses through the Linux network stack. The `XDP_SKB` hook as seen in Figure 2.7 is invoked after such an `sk_buff` has been created and thus replaces the `xdp_md` struct for an `sk_buff`. Created by the NIC driver, the `sk_buff` encapsulates the raw packet data and associated metadata such as pointers to the different protocol headers, the network interface on which the packet arrived, flags, and the length of the packet. It is used by every networking layer from the driver through to IP routing, firewalling, transport layers, and the userspace application's socket. It is designed to offer efficient in-kernel manipulation for operations like adding and stripping headers, and splitting and merging packets.

Listing 2.2: The sk_buff struct

```
type block struct {
        struct sk_buff {
        union {unnamed_union};
        __u16 inner_transport_header;
        __u16 inner_network_header;
        __u16 inner_mac_header;
        __be16 protocol;
        __u16 transport_header;
        __u16 network_header;
        __u16 mac_header;
        sk_buff_data_t tail;
        sk_buff_data_t end;
        unsigned char * head;
        unsigned char * data;
        unsigned int truesize;
        atomic_t users;
    };
}
```

### 2.3.2 XDP Actions

Once an eBPF program has finished running within the XDP hook, it must return one of a few predefined actions, which then instruct the kernel on how to proceed. Table 2.4 shows the six different possible return codes. For instance, if a program's sole purpose is to gather statistics on packets traversing a specific queue and NIC, `XDP_PASS` offers a non-intrusive approach, allowing the packet to continue its journey up the network stack as usual. Alternatively, if a program aims to discard irrelevant packets, perhaps based on a packet's source IP within a Distributed Denial of Service (DDoS) mitigation tool, `XDP_DROP` represents one of the earliest and cheapest ways to reduce the load on the system from malicious packets. `XDP_ABORTED` functions similarly to `XDP_DROP` but with the additional caveat that an error has occurred. For scenarios like load balancing that distribute traffic across multiple machines, `XDP_TX` is highly effective in that it sends the packet back out the same interface it arrived at. Finally, `XDP_REDIRECT` can be used for layer 3 routing or in a special case, sending the raw layer 2 frame directly to userspace, thereby skipping the network stack entirely.

| XDP Action | Description |
|---|---|
| `XDP_PASS` | Allows the packet to continue its journey through the normal network stack for further processing. |
| `XDP_DROP` | Immediately drops the packet at the earliest possible point, preventing it from consuming further resources. |
| `XDP_ABORTED` | Indicates an error or that an exception occurred within the XDP program, leading to the packet being dropped. This is similar to `XDP_DROP` but signals a problem. |
| `XDP_TX` | Transmits the packet back out of the same network interface it arrived on. This is often used for fast replies or packet reflection. |
| `XDP_REDIRECT` | Redirects the packet to a different network interface or to an `AF_XDP` socket in userspace. This enables fast packet steering. |

Table 2.4: XDP Program Return Actions

### 2.3.3 AF_XDP

Table 2.4 highlights the versatility of `XDP_REDIRECT`. Beyond directing packets to other interfaces, it offers a powerful mechanism to send them directly to userspace. This capability is particularly beneficial for performance-sensitive protocols such as QUIC, because it completely bypasses the traditional kernel network stack, thereby eliminating its associated overhead. Achieving this direct delivery requires a userspace application to utilize an `AF_XDP` socket.

### Setting up an AF_XDP Socket

After an application establishes its `AF_XDP` socket through the socket(2) system call, the next step is to set aside a special memory area for incoming packets. This area is called the UMEM region. When creating UMEM, two primary considerations come into play.

First, there's deciding on the size of each "chunk" within this region. Since each chunk holds one packet, it must be large enough for the biggest packet an application anticipates, but not so large that it wastes significant memory. The Linux kernel requires this chunk size to be a power of two, falling between 2048 bytes and the system's page size (which is often 4096 bytes, making 2048 or 4096 byte chunks common).

Second, the total number of chunks must be determined. If this number is too small, there is a risk of the UMEM region filling up and packets being dropped. However, making it too large simply consumes excess memory. Unfortunately, there is no magic formula here; the ideal number of chunks is entirely specific to an application's unique needs.

Once this entire setup is complete, the result is effectively a large, contiguous block of memory that functions very much like an array, ready to receive packets.

Once the UMEM region is configured, the application establishes a communication channel with the kernel (specifically, the eBPF program) using fixed-size, pre-allocated circular ring buffers. This design choice enables highly efficient lockless operations in a single-producer, single-consumer model, as it avoids further memory allocation. However, a direct consequence of this fixed capacity is that when a ring becomes full, new packets are dropped. To therefore ensure data integrity and avoid packet loss, the consumer's processing speed must consistently surpass the producer's rate.

At this point, the userspace process contains ownership over all the chunks. This is represented in Figure 2.8.

Next, to enable seamless packet transfer between userspace and the kernel, four distinct ring buffers are initialized: the `Transmit (TX) queue`, `Receive (RX) queue`, `Completion queue`, and `Fill queue`.

The userspace application then yields ownership of a specified number of UMEM chunks to the kernel, preparing them to receive incoming packets. This is achieved by generating descriptors, each uniquely pointing to a UMEM chunk, and enqueuing them into the `Fill queue` (as depicted in Figure 2.9). Subsequently, the kernel can then retrieve these descriptors and use the associated UMEM chunks to store packets as they arrive from the NIC (this process is illustrated in Figure 2.10).

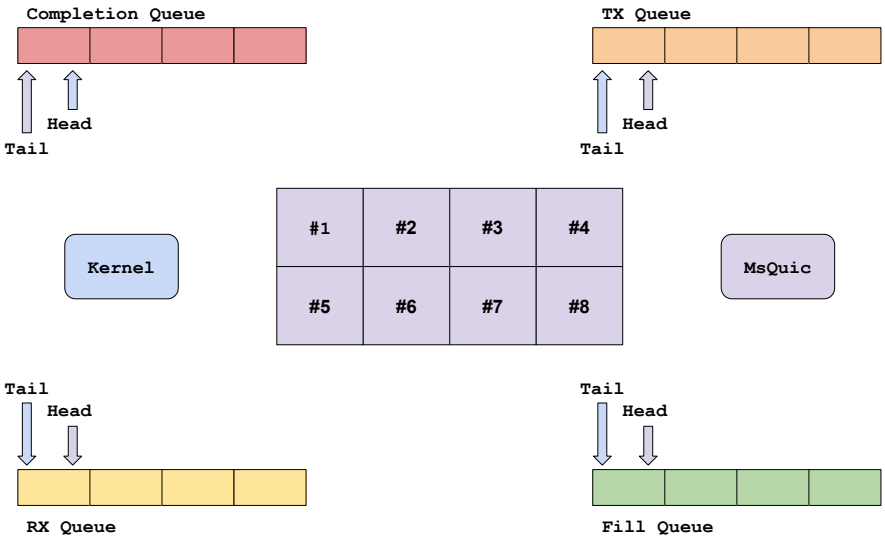



FIGURE 2.8: UMEM After Initialization. Adapted from [15].

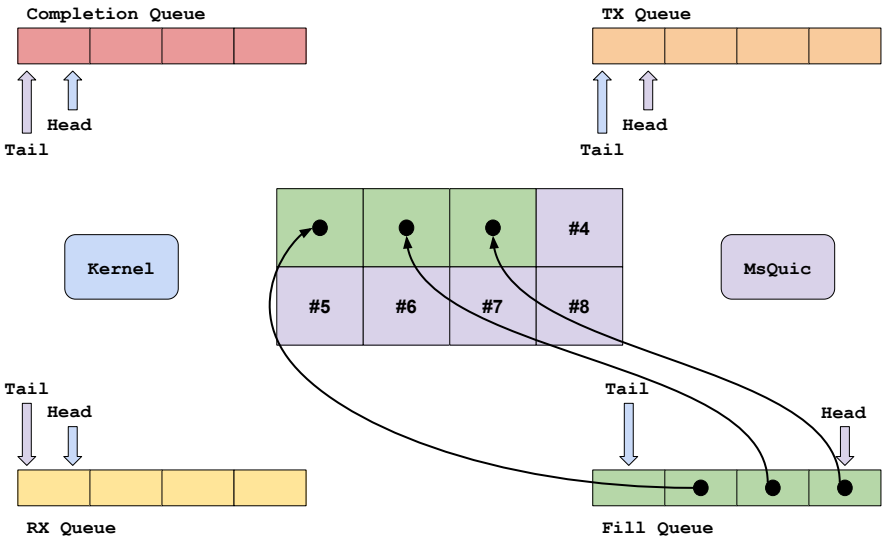



FIGURE 2.9: UMEM Chunks Owned by Fill Queue. Adapted from [15].

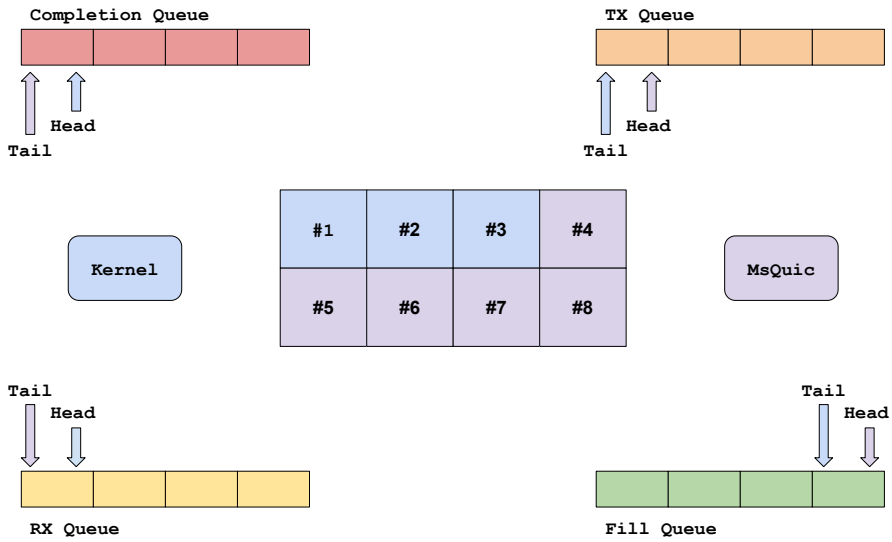



Figure 2.10: UMEM Chunks Owned by Kernel. Adapted from [15].

Once the packets have been copied into the UMEM chunks controlled by the kernel, it moves the descriptors to the `RX queue` as seen in Figure 2.11. The process may then consume these packets.

Once the application has consumed a descriptor from the RX queue, it updates the `RX queue`'s tail pointer and gains two options for handling the associated frame. First, it can return the descriptor to the `Fill queue`, effectively relinquishing control of the frame back to the kernel for future incoming packets. Alternatively, the application can populate the frame with its own data and add the descriptor to the `TX queue`. In this scenario, control passes to the kernel with the explicit intent of transmitting the packet out onto the network interface (as depicted in Figure 2.12). Upon successful transmission, the kernel consumes the TX descriptor and signals completion by adding a new descriptor to the `Completion queue`.

Throughout the lifetime of the process, this interplay of constantly consuming and producing descriptors on the ring buffers enables the transmission and reception of packets with minimal locking and low system call overhead due to the nature of batch sending and receiving the descriptors. Given that packet I/O is the greatest limiting factor to QUIC performance, AF_XDP therefore offers an attractive value proposition with its cheap packet delivery.

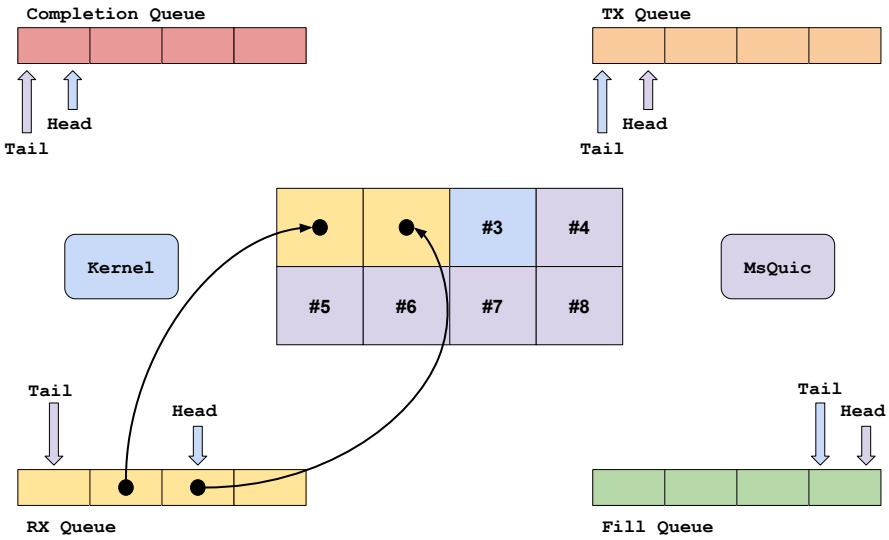



FIGURE 2.11: UMEM after Kernel Adds Descriptors to RX Queue. Adapted from [15].

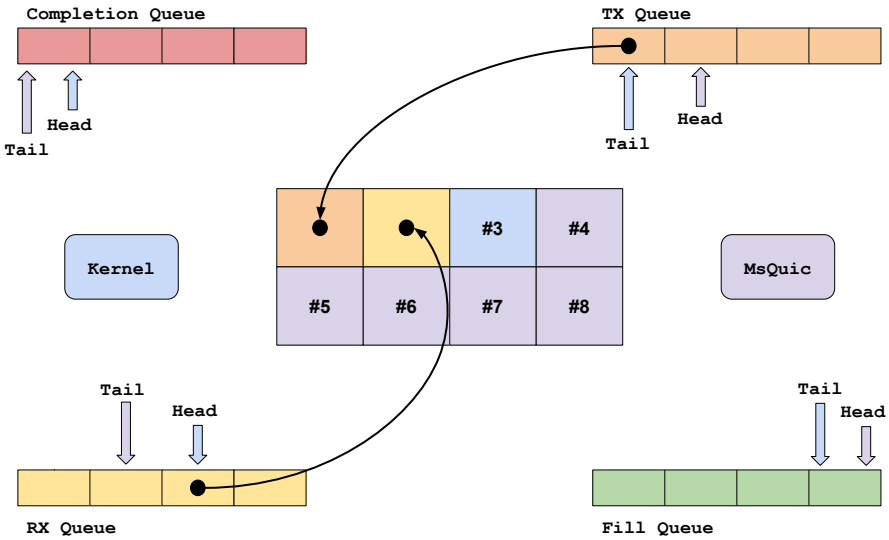



FIGURE 2.12: UMEM after MsQuic Adds Descriptor to TX Queue. Adapted from [15].

### XDP Options

Despite AF_XDP being a relatively new technology, there are a number of available features to change and enhance the way it functions.

#### Zero-Copy

By default, when a packet arrives at the NIC, the traditional kernel receive path involves copying the incoming data into a buffer within the RX ring, as depicted in Figure 2.7. When AF_XDP is active without specific optimizations, the network driver then has to perform a second copy, moving this packet data from the kernel's buffer into a UMEM frame. This double-copy operation introduces noticeable overhead, consuming both CPU cycles and memory bandwidth.

However, this costly double-copy can be entirely bypassed by enabling zero-copy (zc) mode. This is achieved by passing the `XDP_ZEROCOPY` flag to the `bind(2)` syscall when an XDP Socket (XSK) binds with a queue on the NIC. With zero-copy enabled, the NIC is instructed to directly place the incoming Layer 2 packet into a UMEM frame. This means the packet data never needs to be copied into an intermediate kernel buffer, resulting in what should be significantly reduced latency, lower CPU utilization, and higher packet goodput. It is important to note however, that the success of zero-copy depends on the network driver's support and the NIC's capabilities, as it requires direct interaction between the hardware and the UMEM region.

#### XDP Hardware Offload

In the effort to minimize load on the CPU, it has now also become possible to offload an XDP program to the NIC. Pioneered by Netronome, their SmartNIC is able to directly ingest the eBPF bytecode of an XDP program, compile it to the native instruction set, and execute the logic before the packet reaches the host CPU's memory as shown in Kicinski et. al [16]. The implication of such a change is a reduction in latency and an increase in goodput. This is largely due to eliminating the need for costly Direct Memory Access (DMA) transfers into kernel memory and avoiding the traditional hardware interrupt path.

## 2.4 Precision Time Protocol

Throughout history, humans have continuously refined methods of timekeeping to enable synchronization among multiple entities. From sundials to pendulum clocks, the need for precise time has spurred ever-finer-grained models. PTP, defined by the Institute of Electrical and Electronics Engineers (IEEE) as IEEE-1588, is a protocol to synchronize clocks in a computer network [17] and represents the latest step in this evolution. Several

versions exist, each of which supersedes the previous version: PTPv1 (IEEE 1588-2002), PTPv2 (IEEE 1588-2008), and PTPv2.1 (IEEE 1588-2019). PTP is accurate to the sub-microsecond, can function over Ethernet or IP, and is already widely deployed in industry [18].

Setting up a PTP network involves a clear sequence of steps. Initially, each node within the network must be configured. Thereafter, the Best Master Clock Algorithm, as detailed in Subsection 2.4.6, is executed to determine the optimal master clock. The final step is for all nodes to synchronize their clocks, a process elaborated upon in Subsection 2.4.7. It is imperative for a successful synchronization that the nodes align their clock *frequencies*, *phases*, and *times*.

### 2.4.1 PTP Profiles

The PTPv2.1 (IEEE 1588-2019) standard includes various *profiles* that adapt the core PTP protocol to meet the specific requirements of different industries. The PTP topology developed for this thesis utilizes the IEEE 802.1AS profile [19], commonly known as *Generalized Precision Time Protocol (gPTP)*. It is primarily used in Ethernet local area networks for applications such as audio and video, and falls under the Time Sensitive Networking (TSN) umbrella. A key advantage of gPTP is its operation directly at Ethernet Layer 2, which reduces overhead compared to other profiles that function at Layer 3.

### 2.4.2 PTP Tools

Within Linux, three primary tools are used to carry out the clock synchronization in PTP: `ptp4l`, `phc2sys`, and `pmc`. These tools are all part of the *linuxptp* project, which is an open-source implementation of the PTP specification [20]. `ptp4l` is a daemon that synchronizes the PTP Hardware Clock (PHC) of the NIC to match the time of a PTP master clock. It accomplishes this by exchanging PTP messages (i.e. Sync, Follow_Up, Delay_Req, Delay_Resp) with the master clock as further described in Subsection 2.4.7. These messages allow `ptp4l` to calculate the offset, delay, and drift between the slave and master PHCs, all of which are necessary to maintain high accuracy and precision. `phc2sys` is a daemon that disciplines the `CLOCK_REALTIME` (i.e. the system clock) or PHCs with a 'master' PHC. This is desirable for a few reasons. First, `phc2sys` allows the system times of two different computers to be synced with each other, which is crucial for scenarios demanding high temporal accuracy like data logging or event correlation. Second, using `phc2sys` to sync multiple PHCs is advantageous when network topology constraints prevent them from being connected through traditional PTP means. An

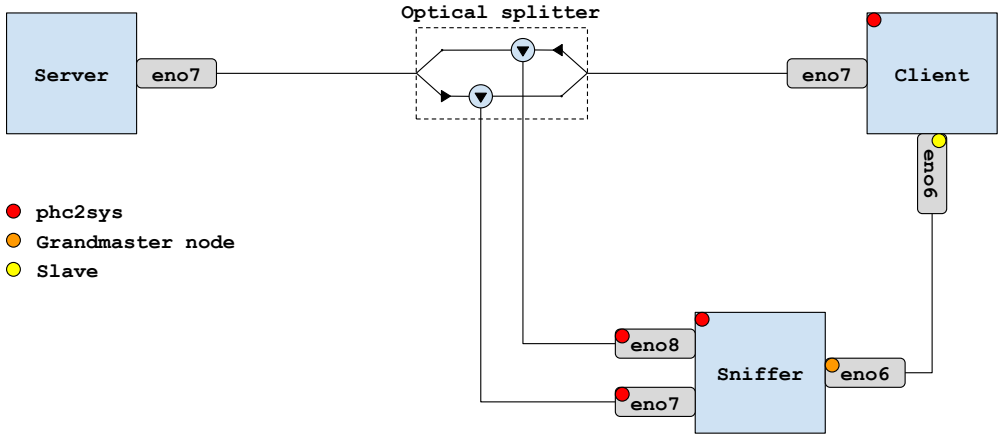



Figure 2.13: A PTP Topology showing `phc2sys` usage. `eno6` on the sniffer disciplines the system time and the PHCs corresponding to `eno8` and `eno7`. `eno6` on the client also disciplines the system time using `phc2sys`. Hypothetically speaking, synchronizing `eno7`'s PHC on the sniffer with `eno6`'s PHC would necessitate a direct link (i.e. additional ports and interfaces) between `eno7`'s and `eno6`'s NICs. This would allow one port on `eno7`'s NIC to continue capturing traffic while the other port is used to discipline the shared hardware clock. The same principle applies to `eno8`.

example of such a topology can be seen in Figure 2.13. Finally, `pmc` is a tool used to configure `ptp4l` during run-time.

### 2.4.3 Time Types

As already alluded to, the concept of "time" itself can be defined and represented in various ways, each with distinct characteristics and applications. While numerous time representations exist, PTP primarily interacts with a select few, each serving different purposes in achieving precise synchronization. This subsection details these relevant time types, from human-centric civil time to highly stable atomic timescales and the internal clocks found within computer systems.

#### Coordinated Universal Time (UTC)

Coordinated Universal Time (UTC) is a timescale tied to the rotation of the Earth and is used for legal, civil, and daily life. Different time zones can be described as offsets from the UTC time. So, for example, Berlin can be represented as *UTC+2*, meaning that it is the current UTC time plus two hours. Unfortunately, one downside to UTC is that because it is tied to the Earth's rotation, which is not perfectly consistent, it is necessary to introduce some form of adjustment in order to keep it in sync with the solar day. This manifests itself in the form of leap seconds. Logically, leap seconds are

problematic for high-precision systems because they introduce discontinuities and make time calculations more complex. Thus, another method is needed.

#### International Atomic Time (TAI)

International Atomic Time (TAI) is an atomic timescale that shares the reference point of January 1st, 1970 with UTC; however, it differs in that it is bound to the atomic scale (e.g. measuring the vibrations of atoms like Caesium-133) as opposed to Earth's rotation. This is beneficial as the behavior of certain atoms are much more precise, which results in an extremely robust monotonically increasing clock. Converting TAI to UTC requires knowing how many leap seconds have occurred since the reference point in 1970. As of July 2025, there have been 37 leap seconds, which means $UTC = TAI - 37$. The benefit is that data centers using synchronized hardware working in TAI are still able to store timestamps in UTC for clients to consume.

#### CLOCK_REALTIME

Most computers have an internal oscillator on the motherboard they use to keep their best approximation of UTC time. The `CLOCK_REALTIME` is a Linux-native label used to interact with this time. When `phc2sys` is used to alter the system time, it references `CLOCK_REALTIME`. This clock is therefore susceptible to PTP.

#### CLOCK_MONOTONIC

Beyond approximating UTC, the internal oscillator also maintains a predictable clock, accessible as `CLOCK_MONOTONIC`. This clock is useful for measuring intervals, with typical use cases including benchmarks, timeouts, and scheduling. It is monotonically increasing, usually commencing at system boot, and is unaffected by PTP adjustments.

### 2.4.4 NIC Hardware Clocks

Besides the oscillator found on the computer's motherboard, some NICs are equipped with their own internal oscillators for timekeeping, as illustrated in Figure 2.14. These PHCs are crucial for achieving extremely accurate timestamping because they are inherently immune to the scheduling delays that can arise with software timestamps in Linux.

To verify whether a NIC supports hardware timestamping, the `ethtool -T` command can be used. A strong indication of hardware timestamping capability, and therefore suitability for PTP, is the presence of *hardware-receive*, *hardware-transmit*, and *PTP Hardware Clock: <num>* in the command's output.

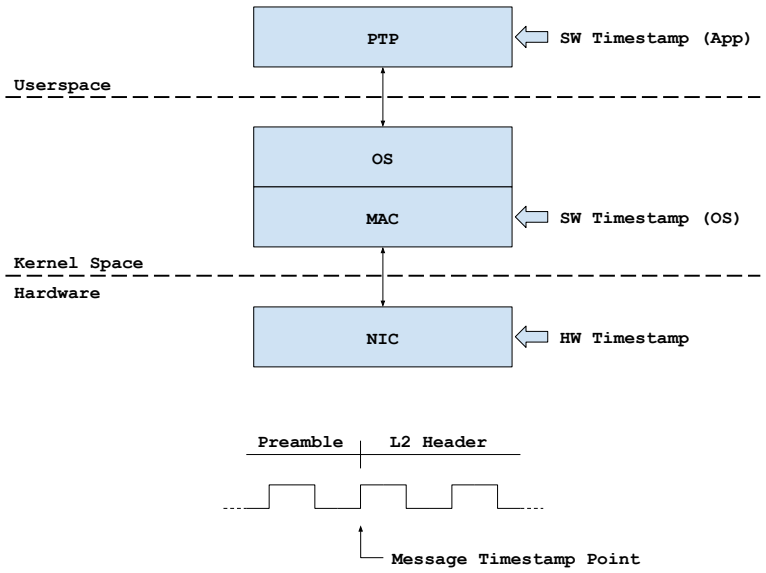



Figure 2.14: PTP Timestamping Options. Adapted from [21].

As a word of caution, it is important to note that some NICs may only offer hardware timestamps up to a certain bit rate or packet rate. Therefore, referencing the NIC documentation is essential to determine these specific limitations.

### 2.4.5 Clock Types

To maintain its high resolution, PTP employs a hierarchy of clocks, each of which fulfills a distinct function. Crucially, no non-PTP-aware devices should exist between PTP nodes, as they can disrupt the extremely delicate synchronization efforts. Furthermore, a tree-topology is always maintained. Redundant ports are automatically switched from 'slave' (S) or 'master' (M) states into a 'passive' (P) state to prevent loops. An example topology is illustrated in Figure 2.15.

#### Ordinary Clock

The Ordinary Clock (OC) is a device type in the PTP network that has a single PTP port and functions either as a source (master) or a destination (slave) with respect to clock synchronization. These devices typically do not participate in the active distribution of time to other network segments. There can be multiple master and slave PHCs in the network.

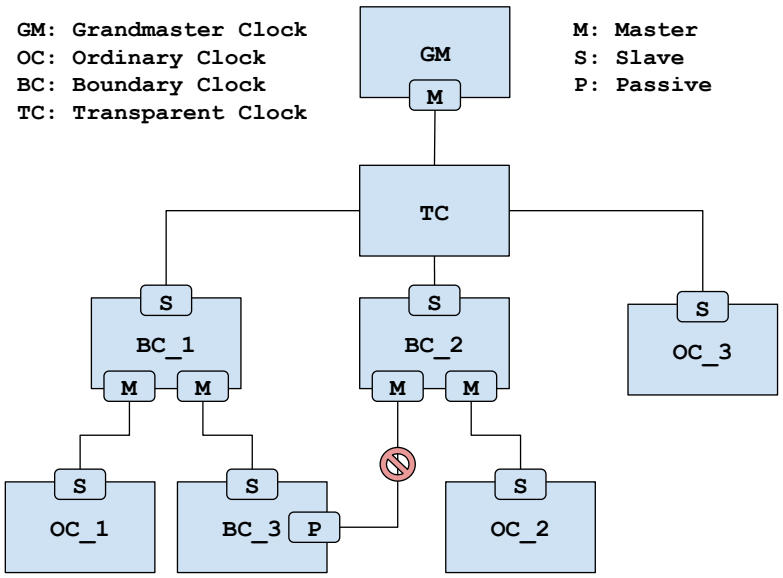



Figure 2.15: Example PTP Network with Pruning.

### Grandmaster Clock

The Grandmaster Clock (GMC) serves as the primary and typically most accurate time source for the entire PTP network. Characterized by its superior clock quality, it often contains direct access to highly precise external references such as a Global Positioning System (GPS) receiver or an atomic clock. A PTP network can only have one GMC at any given time, and every GMC inherently assumes the role of a master clock.

### Boundary Clock

The Boundary Clock (BC) is a device that acts as both a slave to an upstream master clock, synchronizing its own time, and then as a master to downstream slave clocks, distributing that synchronized time to other network segments. It is typically a multi-port, PTP-aware network device, and is often implemented within layer 2 switches. Fundamentally, it exists to scale PTP deployments to more complex topologies and reduce the synchronization load on the master.

### Transparent Clock

The Transparent Clock (TC) is a network device, typically an Ethernet switch as seen in Figure 2.13, whose main purpose is to measure and compensate the time a PTP message spends within the device itself. This 'residence time' is calculated by timestamping the moment of the packet's arrival $t_1$. When the PTP message leaves the TC, the egress timestamp $t_2$ is taken, and the difference between $t_2 - t_1$ is taken and added to the

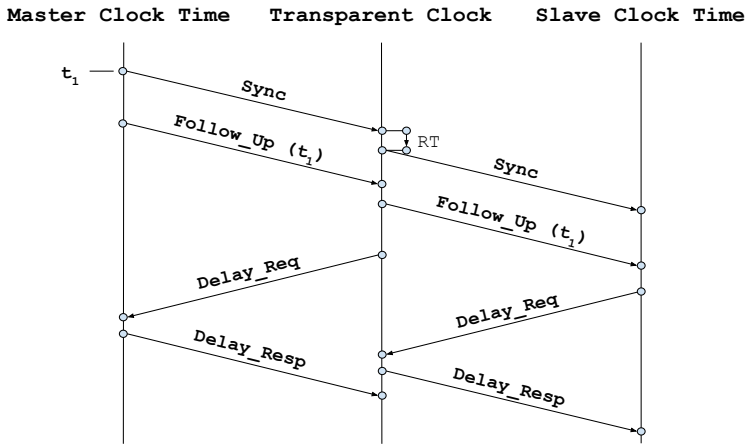



Figure 2.16: Transparent Clock behavior in a P2P PTP Setup. RT is the residence time, which represents the amount of time the message spent within the TC. This value is added to `CorrectionField` as described in Subsubsection 2.4.5. Adapted from [22].

`CorrectionField` within the particular `Sync` message. The slave may then use this field when calculating its clock's offset and delay. This synchronization process is further described in Subsection 2.4.7.

A key advantage of Transparent Clocks is their ability to achieve a flatter PTP topology, contrasting with Boundary Clocks which inherently increase the depth of the tree topology. Also, Transparent Clocks are technically not part of the PTP master and slave state machine the way the other clock types are; however, when used in a Peer to Peer (P2P) setup as seen in Figure 2.16, the TC will additionally calculate the propagation delays of the master and slave links. It will then add the master link delay to the `CorrectionField` as well.

### 2.4.6 Best Master Clock Algorithm

The BMCA runs on each node within the PTP network to establish and maintain a hierarchy of PHCs. To initiate this process, master-node candidates transmit "Announce" messages to their neighbors. By default, these messages are sent every second, although this frequency is configurable, subject to hardware manufacturer constraints [23]. These messages allow the topology to remain dynamic and adapt when changes in the network occur.

Each Announce message carries the properties of the transmitting node's PHC as its payload, and as detailed in Table 2.5, each Announce message includes the follow-

ing attributes: `priority1`, `clockClass`, `clockAccuracy`, `offsetScaledLogVariance`, `priority2`, and `clockIdentity`. When it receives an Announce message, the node checks the advertised `priority1` against its own, using each subsequent attribute as a potential tie-breaker. If the node determines itself as master, it will continue transmitting Announce messages. Otherwise, it changes into a slave and stops transmitting them.

| **PTP Attribute** | **Description** |
|---|---|
| `priority1` | A value (0-255) to set initial preference. A lower value indicates higher priority. |
| `clockClass` | A value (0-255) indicating the clock's quality. Lower values signify higher resolution. |
| `clockAccuracy` | An encoded value representing the clock accuracy to the "true" time. Lower values indicate higher accuracy. |
| `offsetScaledLogVariance` | An encoded value representing the clock's variance. Lower values indicate greater stability. |
| `priority2` | A secondary priority value (0-255), used as a tie-breaker if the listed attributes above are equal. Lower values indicate higher preference. |
| `clockIdentity` | The clock's unique 64-bit identifier. Used as the final tie-breaker in the BMCA. |

Table 2.5: Key Attributes for PTP BMCA

### 2.4.7 Synchronization

Once the PTP topology is established, the PHCs begin their synchronization process. At a high level, a master PHC aims to achieve three key objectives when disciplining its slaves: frequency, phase, and time synchronization. Synchronizing the frequency ensures both clocks tick at the same rate. Synchronizing the phase means the clocks tick at precisely the same instant. Finally, synchronizing the times ensures both clocks agree on the absolute time of day.

Figure 2.17 depicts the message sequence for a slave to achieve frequency synchronization with its master. The master initiates this by dispatching two `Sync` messages and recording their egress timestamps, $t_m^1$ and $t_m^2$, at the moment they exit the NIC. These timestamps are subsequently transmitted to the slave in individual `Follow_Up` messages. In parallel, the slave generates ingress timestamps, $t_s^1$ and $t_s^2$, upon the `Sync` messages'

arrival at its NIC's port. Now equipped with these four timestamp values, the slave possesses enough information to compute its clock drift relative to the master.

The calculation proceeds by first determining the time intervals on both the master and slave sides: $\Delta_m = t_m^2 - t_m^1$ and $\Delta_s = t_s^2 - t_s^1$. Thereafter, the slave's relative frequency drift is computed as Drift $= \frac{\Delta_s - \Delta_m}{\Delta s}$. If this drift value is positive, it signifies the slave's clock is advancing faster than the master's, thus requiring a reduction in speed. Conversely, a negative drift value indicates the slave's clock is lagging, necessitating an increase in speed.

With the frequency synchronized, the slave clock proceeds to synchronize its phase and time-of-day. This is achieved by precisely calculating the propagation delay and the clock offset. The propagation delay represents the time a packet takes to traverse the physical medium from master to slave. The offset is the necessary adjustment to the slave's clock to align it with the master's, after accounting for this propagation delay.

Figure 2.18 illustrates the message exchange for calculating these two values. The master initiates the process by sending a `Sync` message, capturing its departure timestamp, $t_1$, as it leaves the NIC. The master then sends a `Follow_Up` message containing the $t_1$ timestamp in its payload. Upon arrival at the slave, the `Sync` message is timestamped to produce $t_2$. The slave subsequently sends a `Delay_Req` message to the master, recording its departure timestamp, $t_3$, from the NIC. When the `Delay_Req` arrives at the master, a final timestamp, $t_4$ is generated and promptly included in a `Delay_Resp` message sent back to the slave.

Equipped with $t_1, t_2, t_3,$ and $t_4$, the slave can now calculate the offset and delay. We define two intermediate values as seen in Figure 2.18: $A = t_2 - t_1 = \text{delay} + \text{offset}$ and $B = t_4 - t_3 = \text{delay} - \text{offset}$. From these, the propagation delay and offset can be derived as follows: delay $= \frac{A+B}{2}$ and offset $= \frac{A-B}{2}$. As an example, using the values from Figure 2.18, if $A = t_2 - t_1 = 58 - 53 = 5$ and $B = t_4 - t_3 = 64 - 63 = 1$, then the calculated offset is $offset = \frac{A-B}{2} = \frac{5-1}{2} = 2$ and the delay is $delay = \frac{A+B}{2} = \frac{5+1}{2} = 3$. The slave then adjusts its local time by this calculated offset to maintain synchronization with the master.

The synchronization process just described is known as the "two-step setup," due to the separate `Sync` and `Follow_Up` messages. An alternative, "one-step setup," integrates the $t_1$ timestamp directly into the `Sync` message payload, thereby eliminating the need for a separate `Follow_Up` message. By default, `Sync` and `Delay_Req` messages are sent once per second, but their frequency can be adjusted from 8 times per second to once every 2 seconds [23].

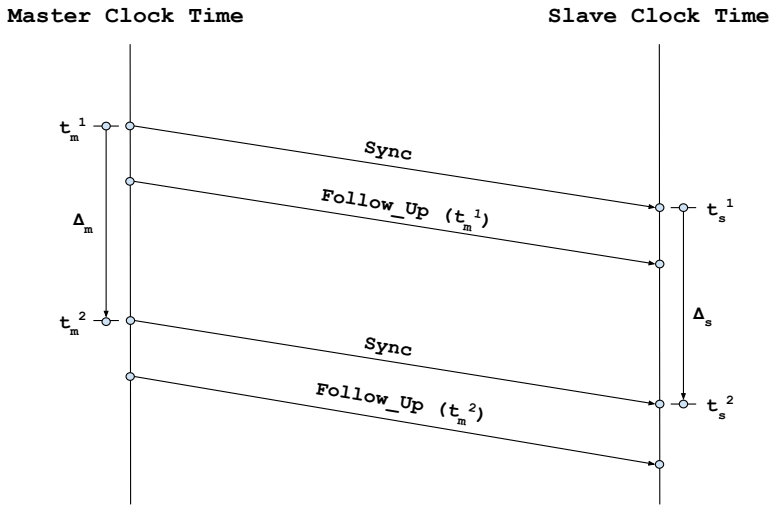



Figure 2.17: PTP message exchange for clock frequency difference calculation using the Peer-to-Peer Method. Adapted from [24].

Finally, it's important to note that Figure 2.18 depicts the Peer-to-Peer model for delay and offset calculation within PTP. An alternative, the End-to-End model, also exists but is excluded here for brevity and because this thesis's PTP topology utilizes the Peer-to-Peer model.

## 2.5 MsQuic

MsQuic[1], Microsoft's QUIC implementation, was selected for the measurements in this thesis due to its high-goodput performance, as demonstrated in Kempf et al. [25]. Furthermore, it is one of the few QUIC implementations that supports XDP. Developed for Windows, Linux, FreeBSD, and macOS, MsQuic employs a two-layered architecture to achieve cross-platform scalability: the QUIC layer and the Platform Abstraction Layer (PAL).

The QUIC layer encapsulates the core QUIC protocol logic and generally remains consistent across different builds. In contrast, the PAL provides abstractions for critical components such as TLS, UDP, and operating system primitives like threads and locks [26]. While this architecture introduces some complexity, it offers the significant advantage of modularity, enabling "plug-and-play" functionality for various components.

[1] https://github.com/microsoft/msquic

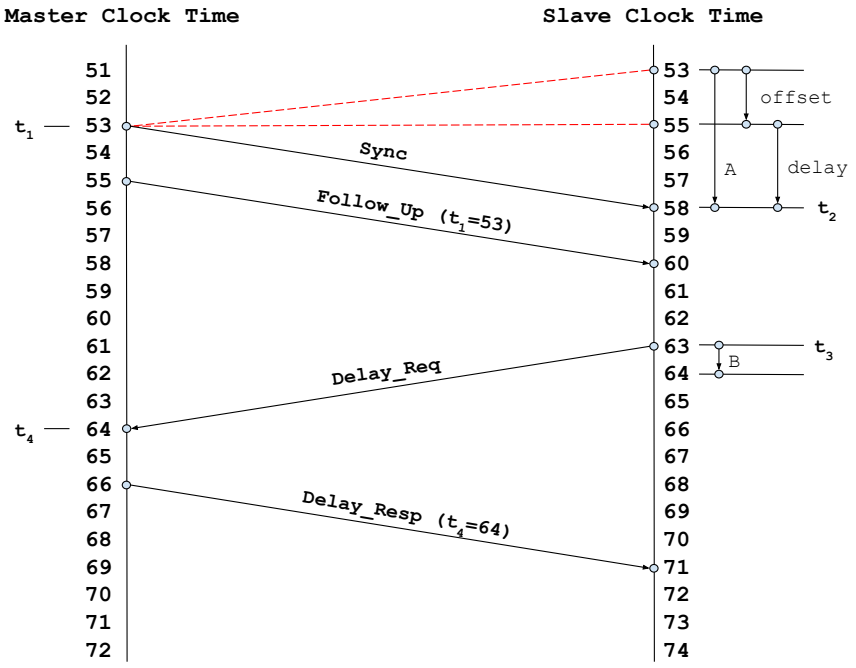



Figure 2.18: PTP message exchange for delay and offset calculations using the Peer-to-Peer Method. Adapted from [24].

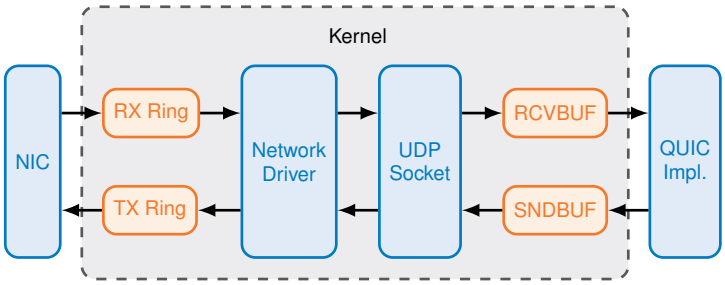



Figure 2.19: Components of the default packet datapath.

For instance, users can choose from three TLS variants: OpenSSL1.1[1], OpenSSL3[2], and Schannel[3].

This modular design also allows for fine-grained control over the datapath, which is where XDP integration becomes relevant. To establish a baseline for comparison with XDP, the subsequent subsections will detail the existing datapath enhancements already incorporated into MsQuic.

### 2.5.1 Receiving Data

Figure 2.19 shows how a packet flows through the kernel to reach a socket buffer, after which a system call such as `recvfrom(2)` can pull the packet's data into userspace. Upon arrival at the NIC, the packet is directly DMA-ed into a buffer within kernel memory, and a descriptor within the RX ring is updated to indicate the buffer contains a valid packet. This is depicted in Figure 2.20. The NIC then signals the Central Processing Unit (CPU) via an interrupt request, indicating packet arrival. In response, a NAPI[4] context is established, which subsequently polls the NIC for further packets.

With the packet now present in the buffer, the NIC driver creates an `sk_buff` structure, which holds references to the packet data in the RX ring, along with essential metadata. This `sk_buff` then traverses the various layers of the network stack, undergoing necessary processing steps, until it is ultimately enqueued into the socket's receive buffer. Finally, the `recvfrom(2)` system call is invoked, triggering a context switch that copies the packet data from the kernel buffer into the userspace application's memory.

[1] https://openssl-library.org/news/openssl-1.1.1-notes/index.html

[2] https://openssl-library.org/news/openssl-3.0-notes/index.html

[3] https://learn.microsoft.com/en-us/windows/win32/com/schannel

[4] https://docs.kernel.org/networking/napi.html

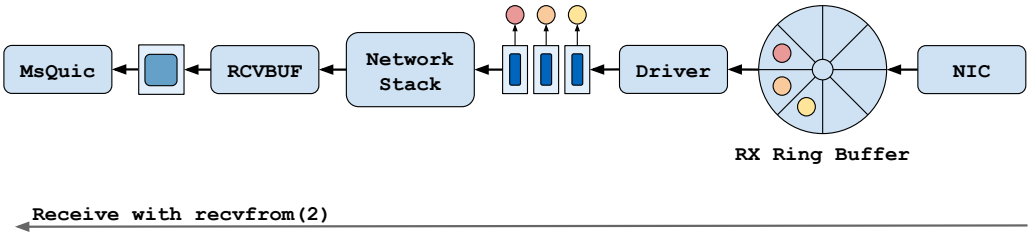



Figure 2.20: Packet Reception with `recvfrom(2)`.

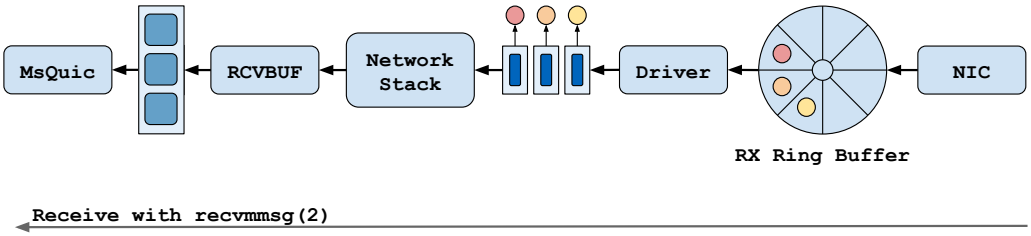



Figure 2.21: Packet Reception with `recvmmsg(2)`.

### recvmmsg(2)

As seen in Figure 2.21, the `recvmmsg(2)` system call represents an improvement over `recvfrom(2)` by allowing an application to receive multiple messages with a single system call. It accomplishes this by providing the kernel with an array of buffers and instructs it to fill as many as possible with available packet data. Fundamentally, this enables the application to *batch* several receive operations into a single system call, which amortizes the cost of the context switch.

### Generic Receive Offload

One downside to the approaches above is that an `sk_buff` is allocated for every MTU-sized packet that arrives on the NIC. Each `sk_buff` then individually goes through each of the network stack processing steps, even if the packets are part of the same flow. To improve on this design, GRO can be used, which assembles packets as part of the same flow under the umbrella of a single `sk_buff`. Given that the currently maximum allowed GRO packet size is 64k bytes[1] and a common MTU size is 1.5k bytes, this can lead to a reduction in processing by a factor of $\frac{64}{1.5} = 43$. As discussed Chapter 4, this

[1] https://elixir.bootlin.com/linux/v6.15.4/source/include/linux/netdevice.h#L2319

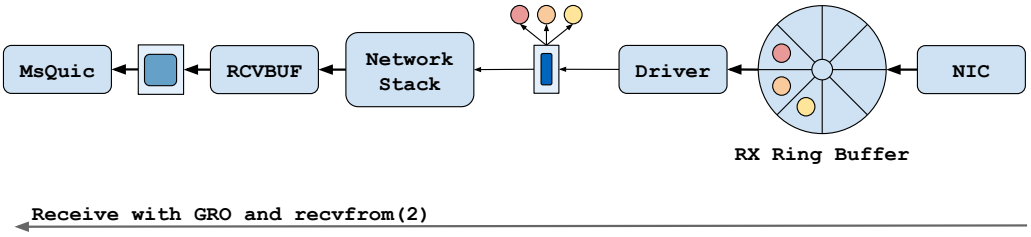



Figure 2.22: Packet Reception with GRO.

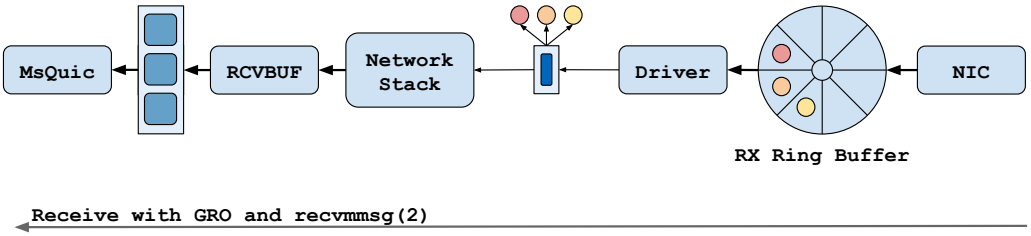



Figure 2.23: Packet Reception with GRO and `recvmmsg(2)`.

improves performance significantly. A representation of this `sk_buff` can be seen in Figure 2.22.

Finally, we can combine these two enhancements together as seen in Figure 2.23. This results in the combination of less overhead per context switch and less processing per byte of data.

### 2.5.2 Logging

MsQuic generates logs and traces using CLOG [27], a cross-platform diagnostic tool developed by Microsoft. On Linux, CLOG seamlessly integrates with LTTng[1], allowing for the production of logs with minimal overhead. Beyond its efficiency, CLOG's messages are highly structured, making them easy to parse for subsequent post-processing. This robust logging capability, as further detailed in Chapter 4, plays an extensive role in the PTP measurements.

[1] https://lttng.org/

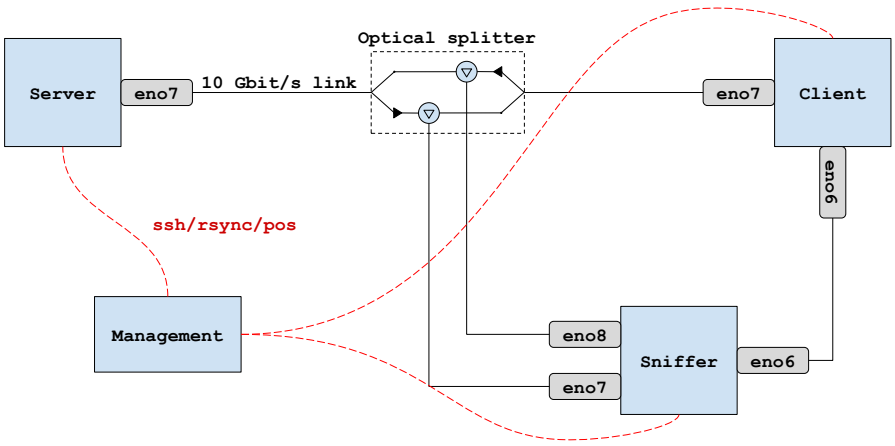



Figure 2.24: Measurement topology of the QIR with dedicated hardware.

## 2.6 pos Measurement Framework

Measurements for this thesis utilized a modified QIR framework, as described by Jaeger et al. [6]. This setup exclusively uses bare metal machines, eliminating potential noise introduced by virtualization software. The network topology for these measurements is detailed in Figure 2.24.

For each measurement, a file-download scenario between a client and server was executed, allowing for goodput inference. A dedicated management node was responsible for resetting machines to a baseline state (e.g., Debian Bookworm) and distributing necessary files and environment variables. Passive optical fiber taps (a.k.a. splitters) were incorporated as non-invasive means to duplicate packets exchanged between the client and server, enabling the sniffer to capture them, which was crucial for PTP tests.

The core advantage of the QIR framework lies in its emphasis on repeatability. It allows for the declarative specification of runtime environments, QUIC implementations, test types, and pre- and post-processing steps, all handled automatically by the QIR.

# Chapter 3

# Implementation

This chapter details the specific changes made to the QIR, the testbed configuration used for our evaluations, and the modifications implemented within MsQuic to enable and optimize XDP.

## 3.1 Testbed Setup

All measurements were performed on bare metal machines within the chair's testbed. For test nodes, we used either the *Intel Xeon D-1518 @ 2.20GHz 4-core* CPU with 32 GB of RAM or the *AMD EPYC 9354 @ 3.25 GHz 32-core* CPU with 768 GB of RAM. To understand how AF_XDP scales with processing power, nodes were specifically chosen from the lowest and highest ends of CPU capability. Additionally, the two *Intel Xeon D-1518 @ 2.20GHz 4-core* nodes were part of the necessary topology for conducting PTP tests, as depicted in Figure 2.24.

To accompany the lower-powered CPUs, the *Intel X552 10 GbE SFP+* NICs with the corresponding ixgbe driver version *6.1.0-17-amd64* were utilized. Their 10 Gbit/s maximum throughput was sufficient to avoid becoming a bottleneck in our evaluations. Crucially, these NICs also offer hardware timestamping support, which is essential for accurate PTP operation.

For the higher-powered *AMD EPYC* CPUs, we used the *Mellanox MT28908 ConnectX-6* NICs with the corresponding mlx5_core driver version *6.1.0-17-amd64*. These were separated by a 100 Gbit/s link.

For clarity, the Intel CPU and NIC configuration will be referred to the "Dogecoin" setup, while the AMD CPU and Mellanox NIC configuration will be known as the "Opulous" setup.

## 3.2 Integrating AF_XDP into the QIR

MsQuic offers two build modes: Debug and Release. Debug mode includes extra features for diagnosing issues, while Release mode, which is designed for higher goodput, omits these. The following subsections detail the steps required to integrate both build modes with XDP in the QIR.

Finally, unless mentioned otherwise, all modifications to MsQuic as part of this thesis were based off version 2.4.7[1].

### 3.2.1 XDP Program Flow

Figure 3.1 illustrates the logical execution flow of the loaded XDP program. For every incoming packet, a series of checks are performed. Initially, the packet is examined to determine if it contains an IPv4 or IPv6 header. If it does, the `ip_map` (an eBPF map object as described in Subsection 2.2.3) is consulted to verify if the destination IP address is a registered entry. If that check passes, the `port_map` is similarly consulted. Should all preceding checks evaluate to true, the RX queue from the NIC on which the packet arrived serves as a key within the `xsks_map` to confirm the presence of a corresponding `AF_XDP` socket. If confirmed, the packet is redirected to userspace using the XDP action `XDP_REDIRECT`. Otherwise, the packet proceeds through the normal network stack via the `XDP_PASS` action. This non-invasive design allows the program to efficiently extract only packets relevant to MsQuic, while ensuring all other packets experience normal network processing.

### 3.2.2 Fixing the Release Build Failure on Debian Bookworm

Before running QIR tests, it was first necessary to build MsQuic and configure it as an artifact for QIR consumption. We chose Debian Bookworm as the build environment, primarily due to its bare-bones nature. However, this immediately led to a significant hurdle. Namely, MsQuic's Release builds on the 2.4.7 version we were using were consistently failing.

[1] https://github.com/microsoft/msquic/pkgs/container/msquic%2Fqns/317647544?tag=v2.4.7

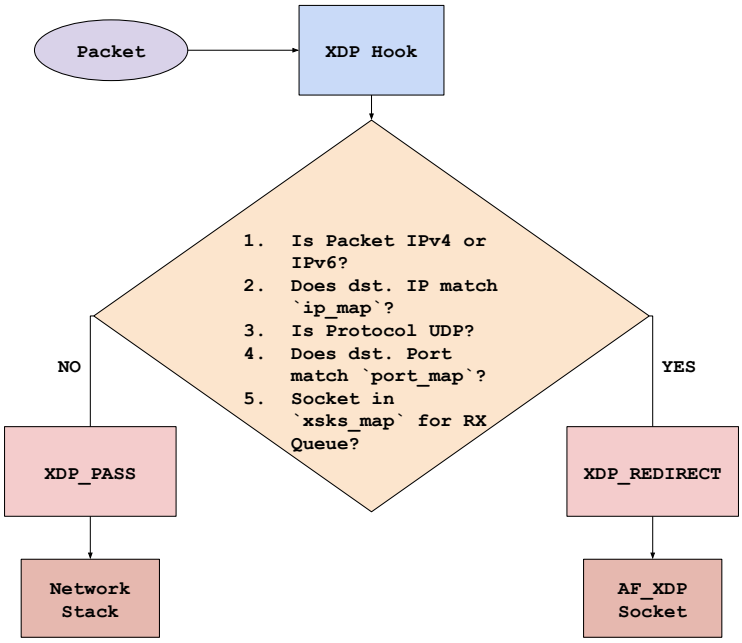



Figure 3.1: XDP Program Flow.

The error messages, a combination of `-Wstringop-overflow=` warnings turned to errors, served as red herrings due to there being no previous GitHub issues on the flagged parts of code. Fortunately, the official MsQuic Continuous Integration (CI) workflow[1] showed successful Release builds on Ubuntu 24.04[2], which strongly suggested the build environment as the root cause. We subsequently identified that Debian's default GNU Compiler Collection (GCC) version was 12.2.0, while Ubuntu utilized 13.2. Manually upgrading GCC on our Debian system resolved the build failures.

To completely eliminate this problem, further investigation showed a specific setting within the MsQuic repository that disabled warnings for `-Wstringop-overflow=` on GCC versions 13 and newer. By adjusting this value to 12 and submitting a pull request[3], we successfully enabled MsQuic Release builds for all Debian Bookworm environments.

[1] https://github.com/microsoft/msquic/actions

[2] https://releases.ubuntu.com/noble/

[3] https://github.com/microsoft/msquic/pull/4828

### 3.2.3 Enabling XDP Within the Client

Although MsQuic provided support for XDP, it did not have the logic integrated into the QIR client and server. Rather, it simply ran as part of an automated test suite. In order to adapt the client to run in the QIR, a number of changes were required. For one, a new execution config was created to include the `QUIC_EXECUTION_CONFIG_FLAG_XDP` flag as seen in Listing 3.1. This forced the interop runner to use the XDP datapath.

Listing 3.1: Enabling the XDP Datapath

```
    ...
    Config->Flags = QUIC_EXECUTION_CONFIG_FLAG_XDP;
    ...
    if (QUIC_FAILED(Status = MsQuic->SetParam(
                        nullptr,
                        QUIC_PARAM_GLOBAL_EXECUTION_CONFIG,
                        BufferLength,
                        Config))) {
        printf("SetParam failed, 0x%x!\n", Status);
```

With XDP now successfully loading, the next steps included enabling transmissions and optimizing the AF_XDP socket and UMEM creation, as introduced in Section 2.3. With respect to enabling the transmissions, a peculiar bug emerged during the first goodput test. Namely, the XDP program loaded correctly, yet failed to initiate transmissions. Using Wireshark[1] on the sniffer (depicted in Figure 2.24), we analyzed the client's outgoing packets. It became evident that the client was sending QUIC Initial packets with a destination MAC address of `00:00:00:00:00:00`. Since the server NIC did not possess this physical address, it strongly suggested the XDP-enabled client was unable to properly resolve the destination's MAC address.

The solution involved correctly resolving the destination IP address (as shown in Listing 3.2) and subsequently setting the corresponding MAC address in the Neighbor Table, which then allowed transmissions to proceed.

Listing 3.2: Hardcoding the IP and MAC address pair within the neighbor table

```
    DEST_IP=$(grep -w $addr /etc/hosts | awk '{print $1}')

    # Update ARP table with retry logic
    max_retries=5
    retry_count=0
    while ! ping -c 1 $DEST_IP; do
    retry_count=$((retry_count + 1))
    if [ $retry_count -ge $max_retries ]; then
        echo "Failed to ping $DEST_IP after $max_retries attempts."
        exit 1
    fi
```

[1] https://www.wireshark.org/

```
12      echo "Retrying␣ping␣($retry_count/$max_retries)..."
13      sleep 1
14      done
15
16      # Extract destination MAC address for $DEST_IP from the neighbor table
17      MAC_ADDRESS=$(ip neigh show $DEST_IP dev $XDP_INTERFACE | awk '{print␣$3}')
18      # Replace entry to neighbor table.
19      ip neigh replace $DEST_IP lladdr $MAC_ADDRESS dev $XDP_INTERFACE
```

### 3.2.4 Optimizing the XDP Datapath

With data exchange now functioning between client and server, a new issue that immediately popped up was the significant amount of time that lapsed between running `run-client.sh` and data actually being transferred.

To diagnose this, we utilized the perf tool[1] to analyze the client's CPU activity during the first three seconds of execution. As highlighted by the purple band in Figure 3.2, the setup of the UMEM region and AF_XDP sockets consumed approximately half of this 3-second trace. A deeper code inspection revealed that XDP was allocating a UMEM region for every single queue across all available NICs on the device. Considering each UMEM region's size of around 100MB and there being multiple NICs present on the client, this led to substantial memory allocation overhead, compounded by the time required for the corresponding `AF_XDP` sockets to be created. This logic was left over from the XDP file's original purpose of running as part of a test suite and was no longer necessary.

Thus, after two simple changes, we were rewarded with a drastic speed-up. The first change we made was using `ethtool` to combine all NIC queues into one. The second involved adding an `-ifindex` flag to the `client.sh` script to specify the desired interface (and its corresponding NIC queue) for binding. This let us bind the eBPF program to a singular queue rather than the many from before. The improved performance, visibly represented by the much smaller purple bar, can be observed in Figure 3.3.

### 3.2.5 MsQuic Release-Mode Goodput Test Failures

During the XDP bring-up, we encountered an additional, perplexing bug. Occasionally, goodput tests using MsQuic Release-mode artifacts would fail. Compounding the issue, these failures produced no discernible error messages, leaving us with minimal diagnostic information. The only clue was that Debug-mode artifacts consistently ran without such failures.

[1] https://perfwiki.github.io/main/

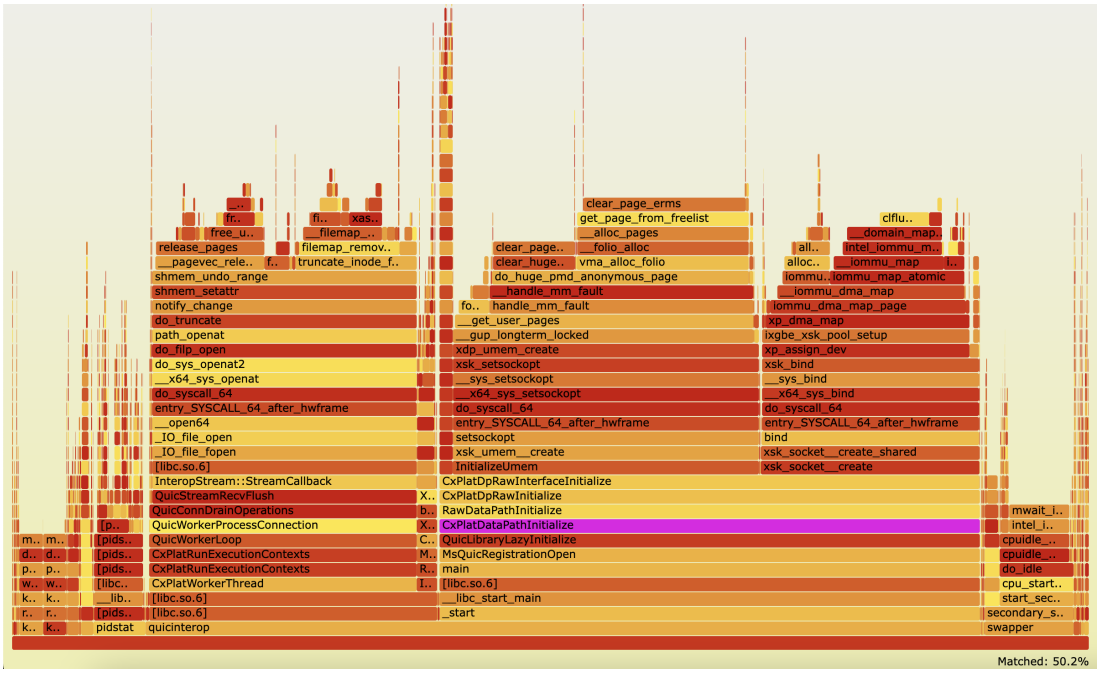



Figure 3.2: A 3-second perf trace was conducted on the XDP datapath initialization prior to optimization. `CxPlatDataPathInitialize`, the bar highlighted in purple, was responsible for approximately 50% of the entire trace, resulting in a delayed transmission.

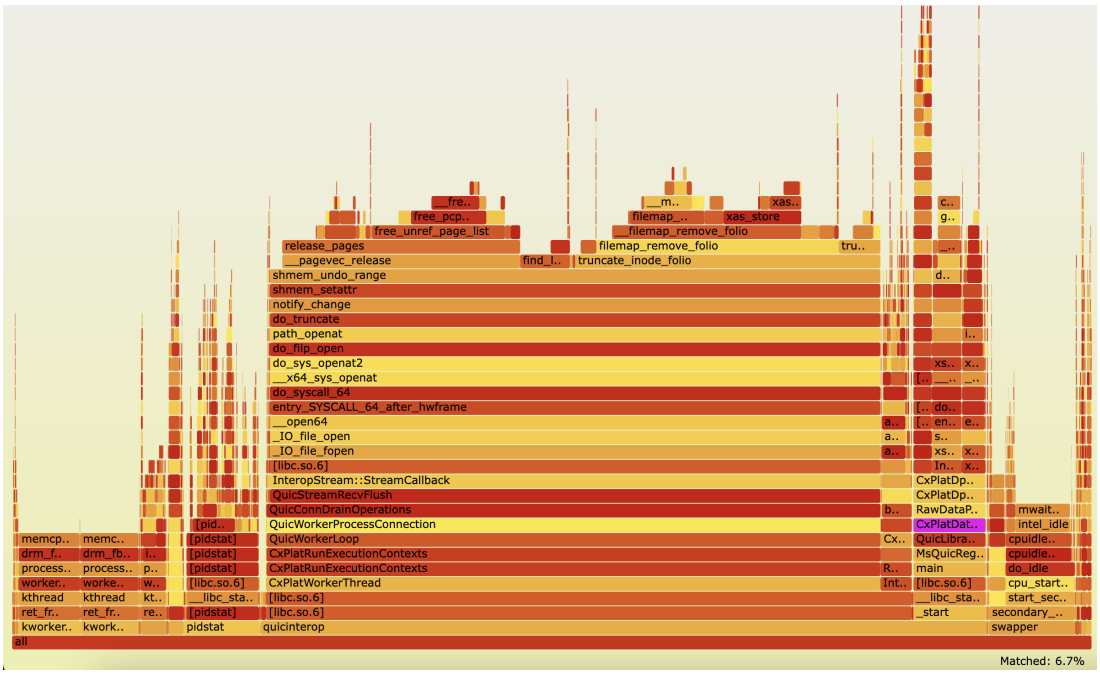



Figure 3.3: A subsequent 3-second perf trace was conducted on the XDP datapath initialization after optimization. The `CxPlatDataPathInitialize`, highlighted in purple, was reduced to 7% of the entire trace, drastically speeding up the connection.

The key difference between Debug and Release modes lay in the Release build's leaner debug content, which resulted in slightly faster transmission times for the same file sizes. This seemingly minor detail ultimately proved to be the crux of the problem. Because the XDP program in Release-mode was encountering packets at a higher rate, it was also consuming the Fill descriptors as described in Subsection 2.3.3 at a higher rate. At certain goodput levels, these Fill descriptors were not replenished quickly enough, leading directly to packet loss. To address this, we increased the number of UMEM frames[1]. This ensured a larger pool of available descriptors, effectively accommodating the higher packet rates in MsQuic's Release mode.

### 3.2.6 Variation of Batch Sizes

Relatedly, one of the tuneable parameters in the userspace XDP logic is the batch size of the RX descriptors[2]. Given the ring buffer of RX descriptors available to userspace, it is possible to extract a variable number of descriptors in a single pass, up to the actual number available at that point in time. We therefore created multiple different variants that adapt the size of the receive buffer: `xdp-drv-zc-batch-{1,2,4,8,16,32,64,128}`. Each number corresponds to the size of the RX batch.

### 3.2.7 Driver Mode, Zero-Copy Driver Mode, and SKB Mode

As described in Section 2.3, there are two XDP hook points: `XDP_DRV` and `XDP_SKB`. Additionally, on supported drivers and NICs, an `AF_XDP` socket combined with `XDP_DRV` can enable the zero-copy option [15]. Figures 3.4, 3.5, and 3.6 illustrate the packet flow from NIC arrival to UMEM copy for skb-mode, driver-mode without zero-copy, and driver-mode with zero-copy, respectively. To evaluate these scenarios, we developed three distinct variants: `xdp-skb`, `xdp-drv-no-zc`, and `xdp-drv-zc`, each corresponding to one of these versions. To analyze each mode, we used the `start-pidstat.sh` and `start-perf.sh` scripts to analyze where the CPU spent its time.

## 3.3 Integrating Precision Time Protocol into the QIR

Integrating PTP into the QIR involved several key steps. Initially, we added LinuxPTP as a Git submodule to the QIR, as detailed in Subsection 2.4.2. This approach allowed for precise version control of the LinuxPTP dependency, which was advantageous for ensuring consistent PTP configurations across all testing environments.

[1] datapath_raw_xdp_linux.c, Line 30

[2] datapath_raw_xdp.h, Line 14

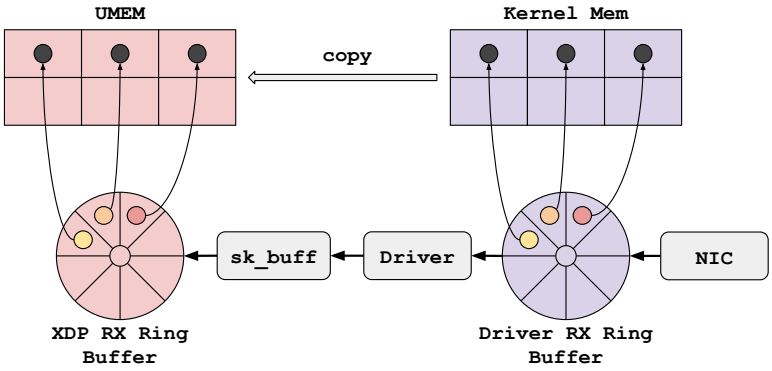



Figure 3.4: SKB mode. The NIC DMAs the packet into kernel memory. After an `sk_buff` is allocated, the packet is then copied again from kernel memory into the UMEM.

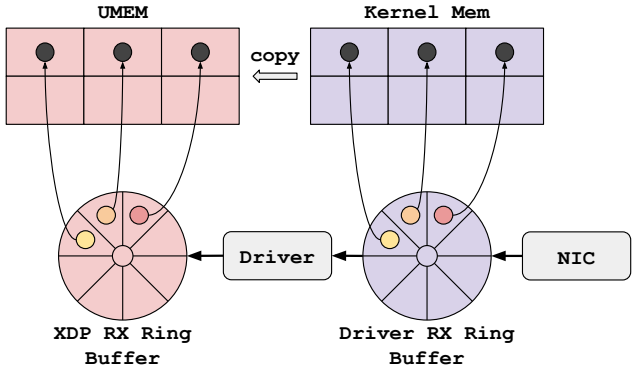



Figure 3.5: Driver mode, zero-copy not enabled. The NIC DMAs the packet into kernel memory, and the packet is copied from kernel memory into the UMEM.

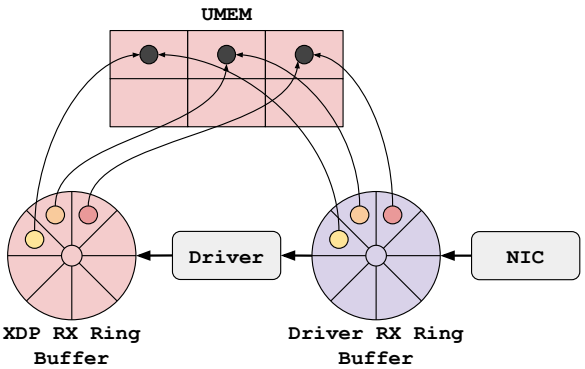



Figure 3.6: Driver mode with zero-copy. The NIC DMAs the packet directly into the UMEM.

Next, we introduced a `-ptp` flag to the `setup-hosts.sh` script within the QIR. When activated, this flag configured each node as usual, and then used rsync[1] to transfer the ptp directory, which contained LinuxPTP and supplementary scripts, to the sniffer and client nodes. Concurrently, as detailed in Subsection 2.4.4, we employed `tc` to limit the throughput of the server's sending interface. This was crucial to ensure the sniffer's NIC could reliably generate hardware timestamps for each packet.

Finally, the `sync-master.sh` and `sync-slave.sh` scripts were executed on the sniffer and client, respectively. These scripts, located within the ptp directory alongside `gPTP-master.cfg` and `gPTP-slave.cfg`, first installed the necessary PTP tools (ptp4l, phc2sys, and pmc) on both machines. Thereafter, these tools were used to initiate the BMCA and begin the clock synchronization processes.

After synchronization and as seen within Figure 2.13, `phc2sys` was used on the sniffer to discipline the system clock (`CLOCK_REALTIME`), as well as two specific interfaces: `eno8`, which consumed the packets routed through the client and splitter, and `eno7`, which consumed the packets routed through the server and splitter. On the client, `phc2sys` was solely used to discipline the system time.

Finally, as a sanity check, LinuxPTP also provides a script `check_clocks` to verify that `CLOCK_REALTIME` is properly disciplined. Since a similar tool was unavailable for the PHCs also being disciplined (specifically, `eno7` and `eno8` on the sniffer as seen in Figure 2.13), we created the script `check_phcs` to do an equivalent check.

### 3.3.1 Measuring the Network Stack Traversal Time

Our fundamental objective in using PTP was to precisely measure the time a packet takes to travel from the NIC to userspace, effectively quantifying its traversal of the 'network stack.' To achieve this, we deployed a `tcpdump` [2] instance on the sniffer to capture messages exchanged between the client and server. For maximum timestamp accuracy, `tcpdump` was configured with the `-j adapter_unsynced` and `--time-stamp-precision=nano` flags, ensuring the use of hardware timestamps generated by the NIC at nanosecond resolution.

As the length of the physical links running from the splitter to the client's and sniffer's NICs were about equal, the sniffer's hardware timestamp served as a reliable proxy for the packet's arrival time on the client's NIC. Furthermore, because the sniffer's

[1] https://linux.die.net/man/1/rsync

[2] https://www.tcpdump.org/

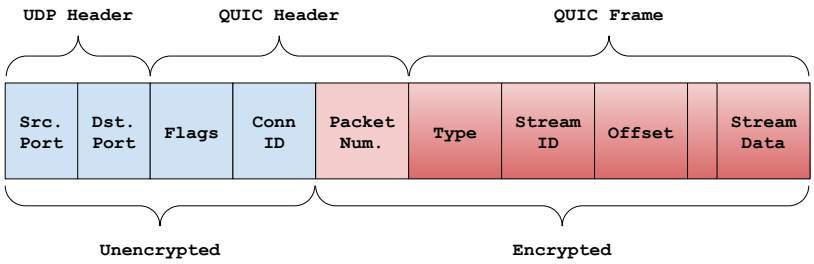



Figure 3.7: QUIC Header. To access the Packet Number, decryption must first take place.

interfaces were disciplined to the same PTP clock as the client's system time, we were confident that the userspace system time used for logs grew in lockstep with the NIC arrival time. We opted for the sniffer-based `tcpdump` approach over running it on the client to avoid introducing additional load that could disturb the measurements.

By using the normal network stack and XDP datapaths, we could evaluate the 'network traversal' times against each other. A crucial prerequisite for this analysis, however, was packet decryption. As seen in Figure 3.7, packet numbers in QUIC are encrypted. This meant a decryption step first needed to occur.

Utilizing the MsQuic logging stack for Linux as detailed in Subsection 2.5.2, we created a new trace event, `LogPacketNumberWithTimestamp`, as seen in Listing 3.3. This event was invoked immediately after packet number decryption[1].

Upon completion of the test, we used `babeltrace`[2] to convert the LTTng data into a human-readable, structured format. Then, to extract only the necessary information from the output, we introduce one more script, `quic.babel.txt-to-json.py`, which pulls the packet number and the timestamp into a JSON file.

Listing 3.3: LogPacketNumberWithTimestamp Trace Event

```
1    QuicTraceEvent(
2            LogPacketNumberWithTimestamp,
3            "[domi] Packet Number: %llu",
4            Packet->PacketNumber);
```

Finally, we integrated two pre- and post-scripts to streamline data collection. The `start-ptp-tcpdump.sh` script handled the initial setup, downloading `tcpdump` and `tcpshark`, and launching two `tcpdump` instances. One instance was used to capture

[1] connection.c, line 4068

[2] https://babeltrace.org/

traffic from the server via the splitter, and the other for capturing the traffic coming from the client via the splitter.

Once data collection finished, the `stop-ptp-tcpdump.sh` script combined these two .pcap files into one. This consolidation was crucial for capturing the entire QUIC handshake, which is essential for the accurate decryption of all subsequent packets. Next, we used `quic-analyzer.py` to process this .pcap file, extracting only the relevant timestamps and packet numbers into a JSON file.

The final step involved correlating these two data sources. To do this, we zipped the LTTng JSON file with our newly generated pcap JSON file. This allowed us to calculate the network stack traversal time by subtracting the userspace timestamp (from LTTng) from the NIC timestamp (from the pcap file).

# Chapter 4

# Evaluation

As detailed in Chapter 3, we used our modifications to the QIR to perform various measurements. In Section 4.1, we analyze the impact on goodput when using different XDP modes (i.e. XDP_SKB, XDP_DRV with no zero-copy, and XDP_DRV with zero-copy (zc)) on both the client and server. We also evaluate the effect of different batch sizes for the AF_XDP RX ring buffer. Following this, Section 4.2 analyzes the packet transit time from the NIC to userspace for both the standard MsQuic datapath and AF_XDP.

For all measurements, except those involving PTP, we conducted 20 iterations of an 8 Gigabyte file download. For the PTP measurements, a single iteration of a 128 Megabyte file download was sufficient, as our packet-level granularity allowed us to sample tens of thousands of packets in one measurement.

Finally, with respect to data representation, for all measurements except PTP, the whiskers on our plots represent one standard deviation from the mean. For the PTP measurements, the whiskers adhere to the 1.5 Interquartile Range (IQR) range. That is, the top whisker extends to the largest data point no more than $Q3 + 1.5 * IQR$, and the bottom whisker extends to the smallest data point no less than $Q1 - 1.5 * IQR$. The IQR itself is defined as $IQR = Q3 - Q1$, encompassing the middle 50% of the data. Q1 represents the 25th percentile, and Q3 represents the 75th percentile. All data values falling outside the whiskers are considered outliers.

## 4.1 XDP Modes

The following subsections detail the implications of using AF_XDP on the client and server.

### 4.1.1 AF_XDP on the Server

Figures 4.1 and 4.2 illustrate the goodput measured with AF_XDP on the servers. In this file-download scenario, the server primarily transmits data, placing a disproportionate load on the TX datapath, which therefore provides a strong proxy for evaluating the TX datapath performance.

Although the AF_XDP configurations do not achieve higher goodputs than the no-XDP case (due to optimizations like Generic Segmentation Offload (GSO) and `sendmmsg(2)`), there is a notable discrepancy in goodput between XDP_DRV zero-copy and both XDP_SKB and XDP_DRV without zero-copy. As the figures demonstrate, these differences become even more pronounced on nodes equipped with higher-performance CPUs.

To investigate why this was occurring, we used pidstat[1] to monitor CPU utilization. Figure 4.3 presents a stacked bar plot of the average CPU utilizations for the system (kernel) and user (MsQuic) on the server during each goodput test. The first important feature to note is that CPU utilization reached 100% for every server-side mode, whereas corresponding client CPU utilizations averaged 90% for DRV zero-copy and 68% for SKB. This clearly indicates the server was the primary bottleneck throughout these tests. More importantly, the SKB and DRV no-zero-copy cases exhibited significantly higher CPU usage in system time compared to the zero-copy case. This further suggests that the low-performing modes were invoking some kernel functions that the zero-copy case was not.

To gain a deeper understanding of the increased kernel time, we took perf traces of the server across multiple tests, resulting in the flamegraphs presented in Figures 4.4 and 4.5. Notably, the `sendto` syscall[2], highlighted in purple, exhibits drastically higher CPU usage in the SKB case compared to the zero-copy case. Specifically, `sendto` consumed 32.8% of the trace's lifetime in SKB mode, but only 15.7% with zero-copy enabled. Considering that the goodputs for SKB and XDP_DRV without zero-copy were lower than XDP_DRV with zero-copy, the corresponding transmission times were

[1] https://man7.org/linux/man-pages/man1/pidstat.1.html

[2] https://linux.die.net/man/2/sendto

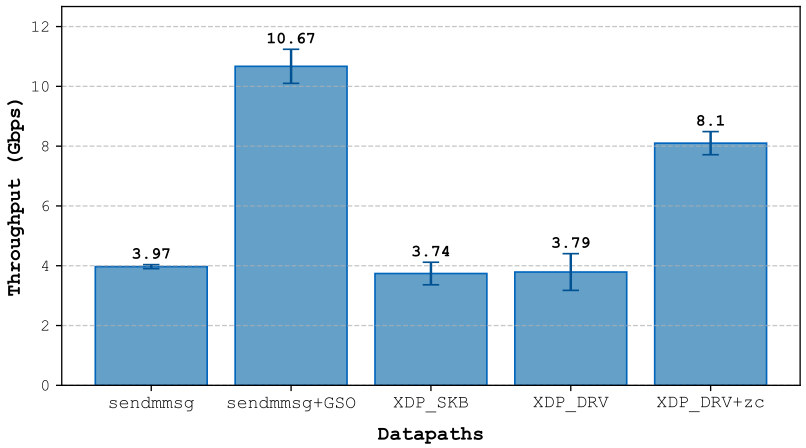



Figure 4.1: A Comparison of the total achieved goodput using different XDP Modes on the AF_XDP server with the "Opulous" setup.

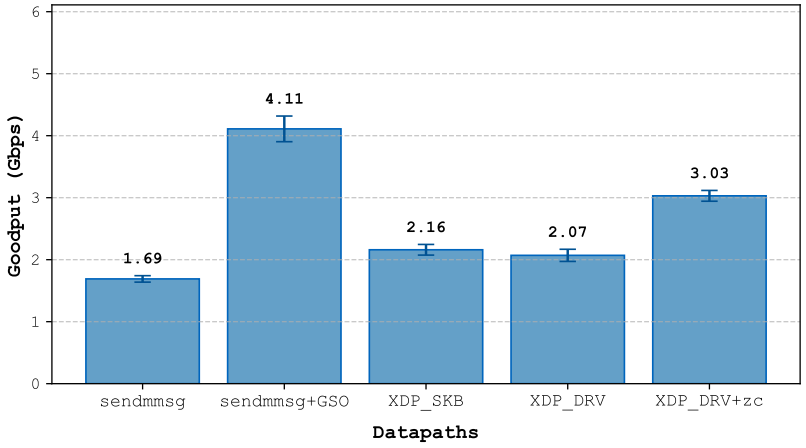



Figure 4.2: A Comparison of the total achieved goodput using different XDP Modes on the AF_XDP server with the "Dogecoin" setup.

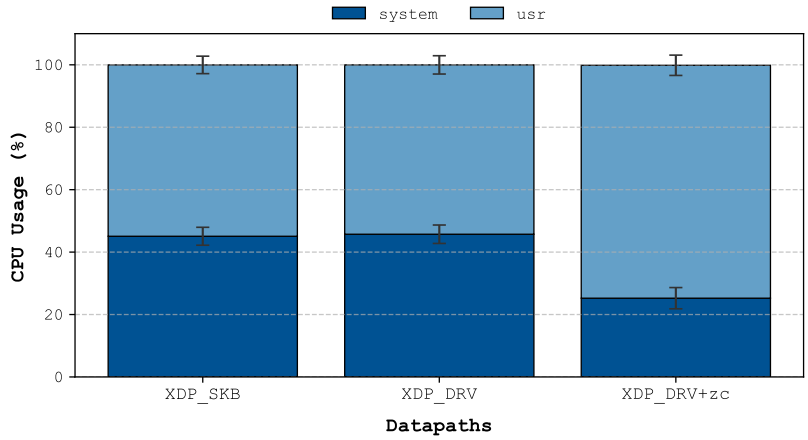



Figure 4.3: A Comparison of the total user and system CPU usage using different XDP Modes on the AF_XDP server with the "Dogecoin" setup.

longer. A larger chunk of a longer transmission time indicates that the `sendto` function had an outsized impact for these two XDP modes.

Drilling further into `sendto`'s call stack, a key divergence between XDP_DRV zero-copy and both XDP_SKB and XDP_DRV no-zero-copy occurs at the kernel function `__xsk_xmit()`[1]. Within this function, a conditional check for whether zero-copy mode is enabled determines the execution path:

- If zero-copy is enabled, `xsk_wakeup()` is invoked, effectively passing control to the driver for packet transmission.
- If zero-copy is not enabled, `xsk_generic_xmit()` is called. This path is computationally expensive because it necessitates copying the packet from the UMEM frame into kernel memory, as depicted in Figure 3.4. Conversely, Figure 3.6 illustrates that zero-copy bypasses this copying step.

Besides the AF_XDP discrepencies, another notable feature within Figures 4.1 and 4.2 is the difference in goodput from disabling GSO. This difference in achieved goodput was so stark that all gains above the AF_XDP datapath were effectively erased. Because GSO effectively batches packets together instead of sending each through the network stack, this difference highlights the true overhead involved with all the networking layers.

[1] https://elixir.bootlin.com/linux/v6.1/source/net/xdp/xsk.c#L597

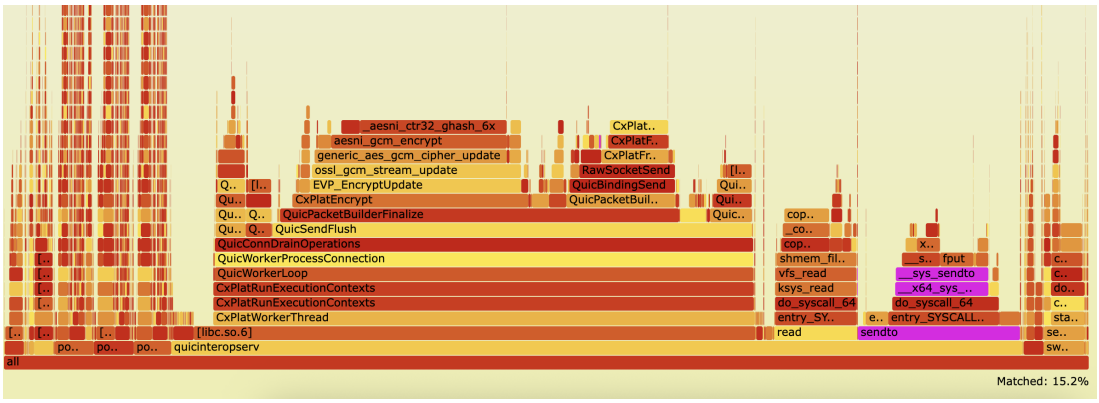



Figure 4.4: Perf trace of the quicinteropserver during a goodput test, utilizing the "Dogecoin" setup with XDP_DRV and zero-copy enabled.

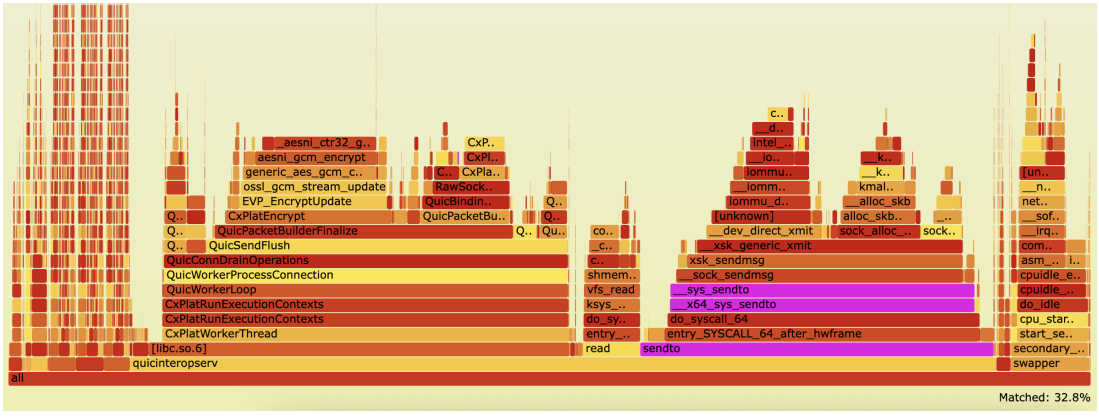



Figure 4.5: Perf trace of the quicinteropserver during a goodput test, utilizing the "Dogecoin" setup with XDP_SKB enabled.

### 4.1.2 AF_XDP on the Client

Figures 4.6 and 4.7 reveal a stark contrast in goodput values for the AF_XDP modes on the client-side relative to their server counterparts. The fundamental difference between having AF_XDP on the client versus the server is that for the client, which receives a disproportionately greater amount of packets than it sends, the XDP receive hook, invoked for every incoming packet, is triggered more often. Consequently, these client-side results serve as a robust proxy for examining AF_XDP's RX performance.

However, the results are surprising as in both high- and low-powered CPU configurations, the goodputs across all XDP modes were not statistically distinguishable. This finding contradicts our initial expectations. As detailed in Subsection 3.2.7, we anticipated that a data copy into the UMEM would be necessary for both the SKB and DRV no-zero-copy modes, implying a performance overhead. Yet, Figure 4.8 shows a strikingly similar distribution of system and user CPU utilization across all XDP modes. If our assumption of a copy in SKB and DRV no-zero-copy cases holds, we would expect a notably higher percentage of system CPU usage, a pattern observed with the AF_XDP server measurements. Unfortunately, this expected increase in system usage was absent. Further attempts to gain clarity through Flamegraph analysis also proved inconclusive.

While a definitive cause for this unexpected similarity in performance across modes remains elusive, one plausible explanation is that zero-copy functionality is not uniformly supported across all NIC queues. As suggested by forum discussions (e.g., in the xdp-newbies mailing list[1]), attempting to enable the ZEROCOPY flag during datapath setup might not actually enabled zero-copy. In our case, we predict the 'zero-copy' XDP_DRV mode to have silently reverted to copying.

Finally, consistent with our analysis of disabling GSO on the server, Figure 4.6 demonstrates that disabling GRO on the client also leads to a dramatic decrease in the total goodput. Without this optimization, the AF_XDP modes achieve higher goodputs than the default network datapath, highlighting the impact of GRO on overall performance.

### 4.1.3 RX Batch Sizes

As detailed in Figure 4.9, the configured batch size for the RX ring buffer can significantly influence the goodput of MsQuic. The diagram shows how the total achieved goodput consistently increases for multiples of 2 until an RX batch size of 32, at which point it begins to plateau.

[1] Re: net/mlx5e: bind() always returns EINVAL with XDP_ZEROCOPY

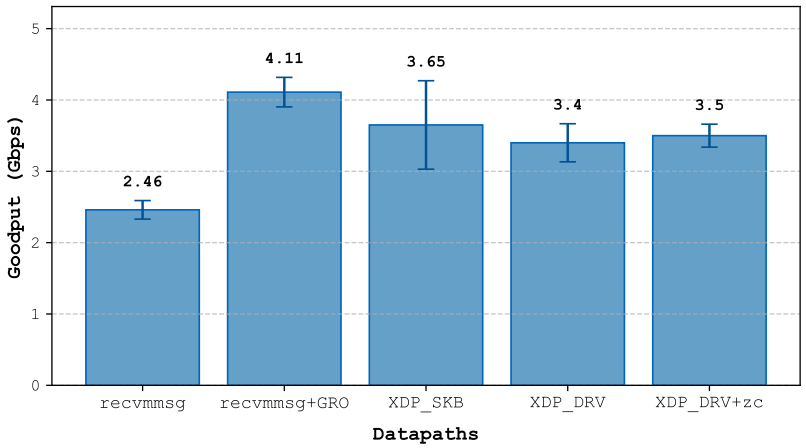



Figure 4.6: A Comparison of the total achieved goodput using different XDP Modes on the AF_XDP client with the "Dogecoin" setup.

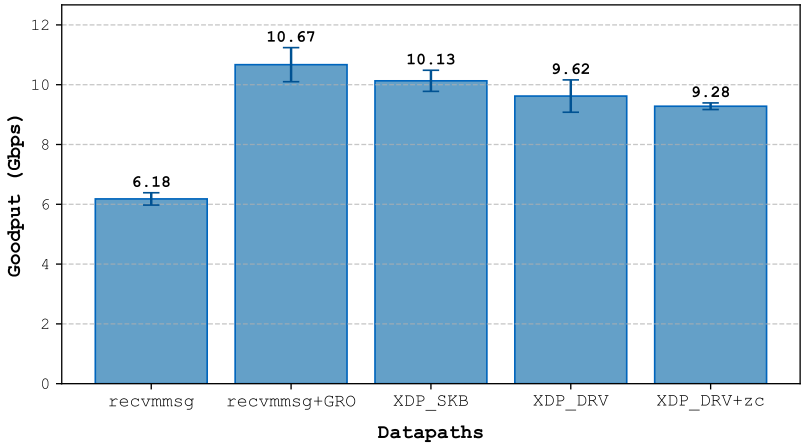



Figure 4.7: A Comparison of the total achieved goodput using different XDP Modes on the AF_XDP client with the "Opulous" setup.

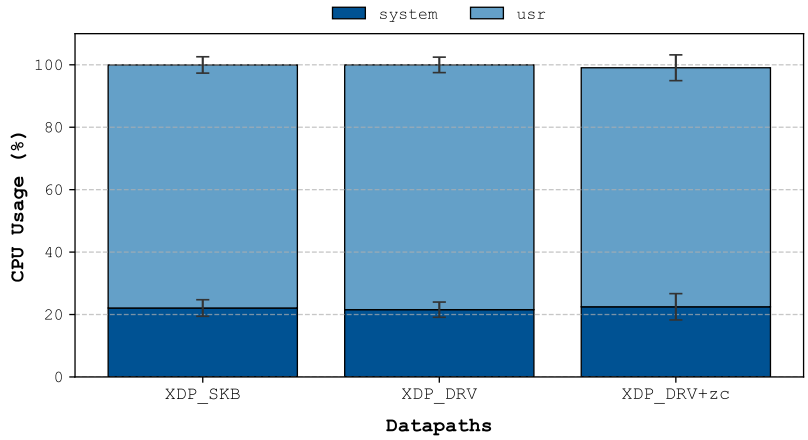



Figure 4.8: A Comparison of the total user and system CPU usage using different XDP Modes on the AF_XDP client with the "Dogecoin" setup.

This observed saturation at an RX batch size of 32 suggests a shift in the application's primary bottleneck. Once MsQuic has fetched a sufficiently large batch of packets, it then performs per-packet operations such as decrypting payloads or stripping and parsing headers. At a batch size of 32, it's plausible that MsQuic consistently receives enough packets to fully saturate the CPU with these processing tasks, eliminating any waiting time for additional packets. Consequently, the batch size ceases to be the limiting factor.

An alternative, or complementary, explanation involves the NAPI_POLL_WEIGHT parameter. As discussed in Chapter 2, a NAPI context is initiated once a packet is DMAed into kernel memory and a hardware interrupt is generated. This NAPI context then begins polling the NIC for new packets. While drivers can technically override it, NAPI_POLL_WEIGHT defaults to sixty-four[1], meaning each poll can process up to 64 descriptors. It is conceivable that from an RX batch size of 32, the MsQuic application reaches a point where it consumes packets at a rate higher than the NAPI context can fetch them from the NIC, leading to the observed goodput stagnation.

## 4.2 NIC to Userspace Packet Traversal time with PTP

To evaluate the relative performance of different datapaths, we measured the time difference between a packet's arrival at the Network Interface Card (NIC) and its subsequent

[1] https://elixir.bootlin.com/linux/v6.1/source/include/linux/netdevice.h#L2547

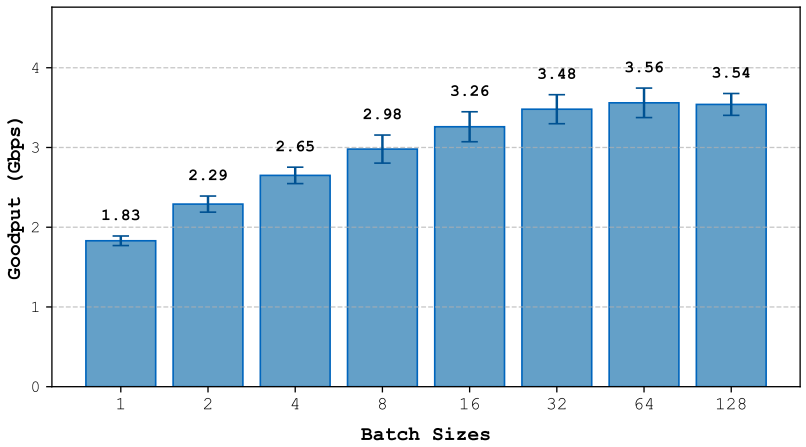



Figure 4.9: A comparison of the goodput achieved with various RX batch sizes in AF_XDP client's zero-copy mode, using the "Dogecoin" setup.

reception within MsQuic in userspace. Figure 2.13 presents two box plots illustrating these differences for both the default and XDP datapaths. The significant overlap in median values, interquartile ranges, and whiskers indicates no statistically significant difference in datapath traversal times between the two approaches. This finding is further supported by Figure 4.11, which displays the Cumulative Distribution Function (CDF) of NIC-to-userspace time differences for each datapath. The only notable distinction is the presence of upper-bound outliers in the non-XDP case. While the precise cause of these outlier delays is unclear, one potential reason could be packets remaining in the socket queue for extended periods.

In summary, despite expectations, the AF_XDP datapath did not demonstrate superior performance compared to the default datapath for this specific measurement. Moreover, further optimization of this particular datapath is unlikely to yield meaningful improvements. This is primarily because network round-trip times are typically measured in milliseconds and increase proportionally with geographical distance between nodes [28]. Therefore, when aiming to reduce latency between networked systems, relocating nodes closer together will have a far greater impact than optimizing the microsecond-level datapath latency within individual hosts.

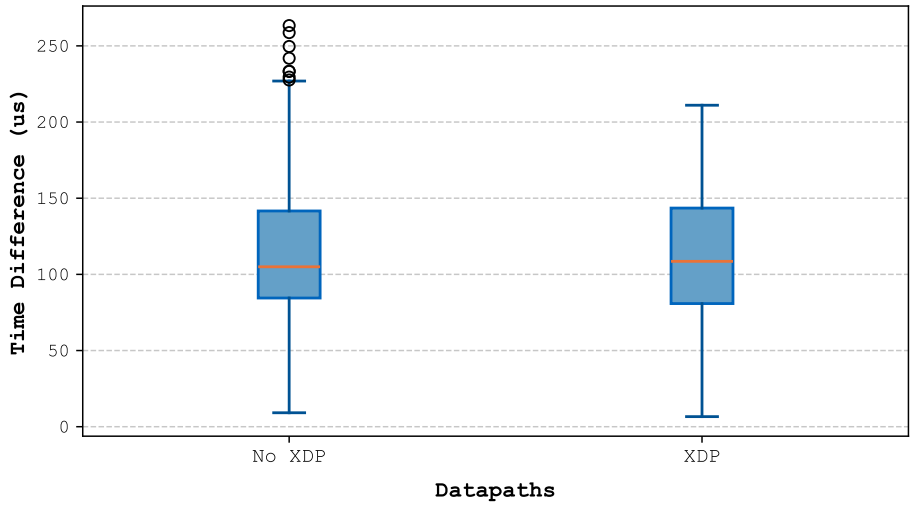



Figure 4.10: Time differences between userspace and NIC hardware timestamps using boxplots and the topology seen in Figure 2.13.

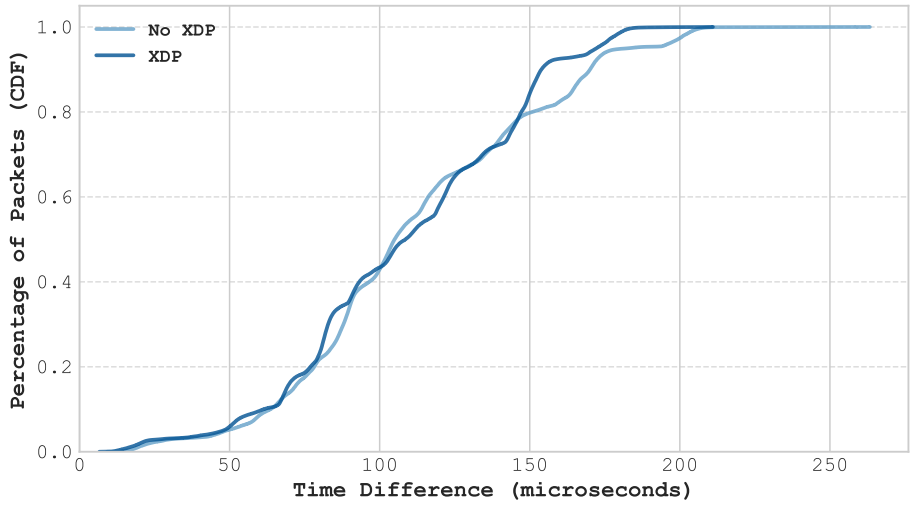



Figure 4.11: Time differences between userspace and NIC hardware timestamps as a CDF plot. The data was generated from a single 128 MB file download test, producing approximately 100,000 packets for this graph. Figure 4.10 is a different representation of the same data.

# Chapter 5

# Taxonomy over QUIC Datapaths

To comprehensively analyze QUIC datapaths, we compare DPDK, AF_XDP, in-kernel QUIC (lxin-quic), and GSO/GRO, while describing the unique tradeoffs inherent to each approach. Although the differences between each datapaths are described in more detail within the subsequent sections, Table 5.1 represents a summary.

Table 5.1: Taxonomy of QUIC Datapaths. Note that the XDP_SKB hook is always supported within Linux while XDP_DRV must be implemented by drivers.

| Metric | DPDK | AF_XDP | In-kernel QUIC | GSO/GRO |
|---|---|---|---|---|
| **Impl. Complexity** | High | High | High | Low |
| **Standardized APIs** | ✗ | ✗ | ✓ | ✓ |
| **Kernel Integration** | Low | Medium | High | High |
| **NIC Sharing** | ✗ | ✓ | ✓ | ✓ |
| **Hardware Agnostic** | Low | High* | High | High |
| **Maturity** | High | Medium | Low | High |
| **Tooling** | Medium | High | High | High |
| **Privileged Access** | ✓ | ✓ | ✓ | ✗ |

## 5.1 Performance Improvement Potential

DPDK fundamentally achieves reduced packet I/O overhead through kernel bypass. Unlike solutions such as AF_XDP's use of ring buffers to minimize system calls and context switches, DPDK completely sidesteps the Linux kernel's networking stack. It moves all packet processing directly into userspace, allowing applications to communicate directly with the NIC. This eliminates the need for frequent system calls and context switches during packet transmissions, significantly boosting performance. To further reduce latencies, DPDK employs a NIC polling driver model instead of an interrupt-based one

and utilizes Hugetables to alleviate pressure on the Translation Lookaside Buffer and to keep related data in contiguous memory.

An in-kernel QUIC implementation offers significant advantages by providing direct access to low-level network stack primitives, such as queueing disciplines. This enables extremely granular control over network traffic, for example, and it represents a level of precision largely unavailable to userspace implementations. Furthermore, the in-kernel approach proves particularly beneficial when utilizing the `sendfile(2)` system call. As illustrated in Figure 2.5, `sendfile(2)` avoids costly data copies into userspace by keeping data exclusively within kernel memory, thereby reducing memory bandwidth and CPU usage.

As detailed in Subsection 2.5.1, GRO and GSO fundamentally reduce per-packet processing overhead. They achieve this by aggregating multiple smaller packets into larger "mega-packets" as data traverses the network stack. This aggregation effectively leads to less processing required per byte of payload data. Consequently, this optimization results in fewer system calls during data transmission and reception, thereby reducing the number of costly context switches and improving overall network efficiency.

## 5.2 Implementation Complexity

Because DPDK offloads all networking functionality from the kernel, integrating it as a QUIC datapath necessitates a complete re-implementation of Layer 2 and Layer 3 packet processing, alongside functionality such as checksum calculation, ARP/Neighbor resolution, and routing. Furthermore, sending and receiving packets would require direct interaction with DPDK's API, which has little overlap with the standard Linux networking API. This makes integration extremely complicated.

Implementing QUIC within the kernel moves most of the protocol's functionality there, while leaving the TLS handshakes to userspace. This inherent split across the kernel-userspace boundary requires synchronization mechanisms such as netlink or the use of control messages with `sendmsg(2)` and `recvmsg(2)`. Additionally, an existing significant challenge for lxin-quic is its reliance on hooking into kernel functions. These functions are inherently unstable across different kernel versions, potentially leading to substantial maintenance efforts to ensure compatibility and stability across various Linux distributions and kernel updates.

A notable disadvantage of AF_XDP is its requirement for extensive knowledge of kernel internals. Developers must have an in-depth understanding of eBPF, XDP hooks, and AF_XDP-specific features such as the four ring buffers and the UMEM region. This

steep learning curve, combined with historically sparse documentation and a limited number of fully-fledged examples, makes AF_XDP integration within a QUIC implementation also challenging.

Implementing GSO and GRO within QUIC is comparatively straightforward. It primarily involves leveraging specific kernel socket options and system calls. Relative to the other datapaths discussed, they require minimal setup.

## 5.3 Unwanted Side Effects

A significant drawback of using DPDK is its exclusive control over the NIC. By detaching the NIC from the kernel, DPDK renders it "invisible" to the standard Linux networking tools and other applications. This is particularly problematic in single-NIC systems, as it prevents other userspace applications from accessing the network. Furthermore, DPDK's hardware configurations are limited, as it only supports a specific set of NICs and drivers[1].

The appropriate size of the UMEM region within AF_XDP is highly dependent on the system's total throughput. As detailed in Subsection 3.2.5, reserving too small a region can lead to silent packet loss, impacting application reliability.

Bugs within an in-kernel QUIC implementation pose a much greater risk than those in a userspace implementation. An in-kernel bug can lead to a system-wide crash, affecting all running applications. Conversely, a bug in a userspace QUIC implementation would typically result in only a single application failure, limiting the scope of impact.

One notable downside to GSO is the potential for packet bursts upon transmission. Because packets are "jumbled" together into a large frame until the very last moment in software, there are often few mechanisms to prevent these bursts as they leave the NIC, as demonstrated by Kempf et al. [29]. This unmitigated bursting can lead to performance degradation.

## 5.4 Privilege Level Usage and Deployment

Bringing up lxin-quic requires loading it as a kernel module. Like other kernel modules, this operation necessitates root privileges or the `CAP_SYS_MODULE` capability[2]. With

[1] https://core.dpdk.org/supported/#nics

[2] https://man7.org/linux/man-pages/man7/capabilities.7.html

respect to deployment, lxin-quic aims for smooth integration by adopting the standardized Linux socket APIs. This means applications already using the Linux socket API for TCP can leverage the same networking system calls (e.g., `listen(2)`, `accept(2)`, `connect(2)`, `sendmsg(2)`, and `recvmsg(2)`) for QUIC. This significantly reduces the friction associated with its adoption. Finally, it's important to note that lxin-quic is currently under active development and is in an immature state, which may lead to trouble during deployment.

Because DPDK also relies on kernel modules such as `vfio-pci`, it too requires elevated privileges for use. For deployment, Hugepage support must be enabled within the running kernel via the HUGETLBFS option[1]. Once the DPDK source code is downloaded, it is compiled, and scripts are used to bind the NIC to DPDK. This binding grants the application complete control over sending and receiving network traffic, after which it may begin to send traffic.

XDP also requires higher privileges. Specifically, loading XDP programs (of type `BPF_PROG_TYPE_XDP`) is a privileged operation. For deployment, a library like libxdp[2], which contains XDP helper functions, may be used. At runtime, the application loads the eBPF program into the XDP hook, which then redirects packets to userspace. Subsequently, the application must create the AF_XDP socket and User Memory (UMEM) region as described in Section 2.3.

GSO and GRO must first be enabled using a tool such as `ethtool -K`[3]. Within the context of UDP, the desired segment size (i.e. UDP_SEGMENT) must then be specified with a `setsockopt` call. This allows the userspace application to send a "massive" packet through the network stack, which is subsequently split into smaller segments in software just before transmission.

[1] https://doc.dpdk.org/guides/linux_gsg/sys_reqs.html

[2] https://github.com/xdp-project/xdp-tools/tree/main/lib/libxdp

[3] https://man7.org/linux/man-pages/man8/ethtool.8.html

# Chapter 6

# Related Work

In this chapter, we provide an overview of existing literature relevant to QUIC performance analysis and optimization and describe how our work differs.

### QUIC Performance Improvements Using Socket Optimizations and Kernel Bypass

This thesis is thematically similar to the Master's Thesis *QUIC Performance Improvements Using Socket Optimizations and Kernel Bypass* [10], which investigates methods to improve QUIC performance by means of patching MsQuic with DPDK and using socket optimizations such as segmentation offloading and message batching. As a result of his experiments, Spaeth showed that the default MsQuic datapath using optimizations such as message batching (i.e. `sendmmsg(2)`) and segmentation offloading (i.e. `GRO/GSO`) was able to reach a goodput of 8254 Mbit/s, which outperformed the TCP+TLS reference stack. Moreover, Spaeth showed that kernel bypass with DPDK reached a goodput of 4290 Mbit/s, which was not able to exceed the goodput of the default datapath.

This thesis differs from Spaeth's work in that it focuses on goodput measurements using XDP rather than DPDK. Additionally, this thesis placed an additional emphasis on latency by measuring the 'network stack' traversal time of packets using PTP with various datapaths. One similarity between our work, however, is that both DPDK and XDP were not able to exceed the goodput of the default MsQuic datapath.

### Improving the performance of picoquic by bypassing the Linux Kernel with DPDK

This thesis is also thematically similar to the Master's Thesis *Improving the performance of picoquic by bypassing the Linux Kernel with DPDK* [30], which investigates the performance increases resulting from adding DPDK to picoquic [31], a QUIC implementation written in C. From his experiements, Tyunyayev showed how DPDK increased goodput by a factor of three relative to the default datapath in a simulated web request test when the number of cores was limited to one. With the core-contraint lifted, the performance difference between DPDK and the default datapath reduced to 1.85x. These results, being contradictory to those by Spaeth, can be partly explained by the fact that picoquic used an unoptimized default datapath. For example, one such feature not included by picoquic was message batching (i.e. `recvmsg(2)` instead of `recvmmsg(2)`). With respect to latency, Tyunyayev also showed the DPDK version having a 30% lower latency over the default datapath.

Much like with Spaeth, this thesis differs from Tyunyayev's work in that it focuses on goodput measurements using XDP rather than DPDK. Additionally, this thesis focused on a slightly different type of latency by measuring the 'network stack' traversal time of packets using various datapaths with PTP, rather than the time-to-first-byte of web responses as done by Tyunyayev.

### Accelerating QUIC with AF_XDP

Huang and Zhao [32] investigated quic-go[1] by integrating `AF_XDP` as a new datapath. They developed `XSKConn`, a Go library that replaces the standard Linux network stack datapath with an `AF_XDP`-based alternative. Their throughput and Requests per Second (RPS) tests demonstrated that the XDP-enabled quic-go, both with and without zerocopy, consistently handled more RPS than the default datapath in ping-pong and file-download scenarios. Notably, with more than four clients, `XSKConn` improved RPS by over 30% in the ping-pong tests. Furthermore, the XDP versions exhibited significantly lower CPU usage (5-50% less) in both test cases.

This thesis diverges from Huang and Zhao's work in several key aspects. First, we focus on MsQuic, which has demonstrated superior goodput compared to quic-go [10]. Additionally, our research uniquely measures network traversal time using PTP, a metric not explored by Huang and Zhao. Finally, contrary to their findings, our throughput tests revealed inferior performance with XDP compared to the default MsQuic datapath.

[1] https://github.com/quic-go/quic-go

This discrepancy could stem from Huang and Zhao's measurements being conducted on an older version of quic-go that lacked datapath optimizations like GRO[1], which MsQuic already included.

#### LXIN-QUIC

Already introduced within Chapter 4, lxin-quic [9] is an active project bringing a majority of QUIC features into the kernel, while keeping the handshake within userspace. It establishes a new socket type `IPPROTO_QUIC`, which operates over UDP tunnels. Fundamentally, this approach offers a number of benefits over application-space QUIC implementations. First, it adhears itself to the standardized networking APIs by covering operations like `listen(2)`, `accept(2)`, `connect(2)`, `sendmsg(2)`, `recvmsg(2)`, `close(2)`, `get/setsockopt(2)`, and `getpeername(2)`. This makes it easier for applications using TCP to switch over to QUIC. Another advantage is the minimization of data duplication through system calls like `sendfile(2)` as described in Subsection 2.1.7. Although existing outside the kernel during its development[2], lxin-quic has now been included as part of a patch set introducing the core QUIC infrastructure and subcomponents[3].

This thesis differs from lxin-quic for the clear reason that our work with MsQuic represents a fundamentally different approach at integrating QUIC, being that it exists in userspace. XDP and DPDK are therefore incompatible with the lxin-quic approach.

[1] https://github.com/quic-go/quic-go/issues/3838

[2] https://github.com/lxin/quic

[3] https://lwn.net/Articles/1028932/

# Chapter 7

# Conclusions and Future Work

In this chapter, we conclude our work. We begin by answering our research questions, then summarize our findings and outline future work.

**To what extent can we increase QUIC performance by using XDP in a trivial way? Is there optimization potential?** Our evaluation in Chapter 4 reveals that AF_XDP, in its current implementation, underperforms the default network datapath in goodput for both receiving and sending-dominanted scenarios. Furthermore, AF_XDP does not offer a statistically significant reduction in the NIC-to-userspace packet traversal time.

Despite this, however, our analysis highlighted significant performance variations among the different XDP modes (i.e. XDP_DRV without zero-copy, XDP_DRV with zero-copy, and XDP_SKB). Specifically, within the server's AF_XDP sending datapath, we demonstrated how the additional memory copy from the UMEM frame into kernel memory, inherent in XDP_DRV without zero-copy and XDP_SKB, drastically reduces goodput.

Finally, as detailed in Subsection 7.0.1, further optimizations to the AF_XDP datapath are possible, which could make it more competitive against its alternatives.

**Can a taxonomy be created for QUIC optimizations, categorizing them by implementation complexity, performance improvement potential, unwanted side effects, privilege level usage, and deployment?** Additionally, within Chapter 4, we compare four distinct QUIC datapaths: lxin-quic, DPDK, AF_XDP, and GSO/GRO. We analyze each based on their unique strengths and weaknesses with respect to the different categories. For instance, while DPDK eliminates the over-

head of the Linux network stack and avoids user-kernel space data copies, it requires DPDK-specific driver and tool implementations, significantly increasing complexity for the user. Conversely, lxin-quic's adherence to standardized Linux networking APIs such as `sendmsg(2)` and `recvmsg(2)` makes it easy to adopt; however, it is a largely immature implementation undergoing very active development. Finally, AF_XDP, despite being a truly Linux-native implementation, currently lags in performance compared to DPDK [33] and the default Linux network datapath with optimizations (i.e. GSO and GRO).

### 7.0.1 Performance Improvements

Throughout the research for this thesis, we identified several promising ideas for future investigation. The following sections dive into some of these ideas.

#### Enabling GRO within XDP

The performance advantages of GRO, primarily its reduction of per-packet overhead within the Linux network stack (as shown in Subsection 4.1.2), can be similarly extended to XDP. As written in Bianconi et al. [34], initial efforts towards this integration are already underway, and XDP Fragments, which is a required component to support GRO in XDP, is gaining support across an increasing number of drivers [35]. As seen in Figure 4.6, the difference between including and not including GRO makes a dramatic impact to the default datapath, so the gains to XDP could offer similar upside.

#### Increasing the TX Batch Size

One promising modification to MsQuic for improved transmit (TX) performance involves increasing the TX ring's batch size. In MsQuic version 2.4.7, which was used for our measurements, the `CxPlatDpRawTxEnqueue` function currently reserves only a single descriptor from the TX queue for each data transmission[1]. Evidence from Karlsson et al. [33] shows that larger TX batch and ring sizes significantly boost throughput in 64-byte packet benchmarks. Moreover, given that the kernel supports a maximum TX batch size of thirty-two[2], similar gains to those present in the RX batch size increases as illustrated in Figure 4.9 could be realized.

[1] datapath_raw_xdp_linux.c, line 1141

[2] https://elixir.bootlin.com/linux/v6.1/source/net/xdp/xsk.c#L33

### Expanding NIC Hardware Offloads

As highlighted by Jaeger et al. [36], encryption and decryption are the second largest consumers of CPU resources after packet I/O. Consequently, enabling hardware offload for these tasks would significantly impact performance. MsQuic is currently focused on enabling such offload capabilities[1], with a proposal already describing the necessary steps for hardware vendors to offer support[2]. Exploring this area could thus be a valid approach to improving goodput.

Finally, as detailed in Yang et al. [37], hardware offloading can be pushed yet even further to also include features such as packet reordering and checksum offloading. Fundamentally, the more work a NIC takes off the CPU, the more cycles the CPU will have for other processes.

[1] https://github.com/microsoft/msquic/issues/4780

[2] https://github.com/microsoft/net-offloads/blob/main/quic-encryption-offload.md

# Chapter 8

# Appendix

**0-RTT** 0 Round Trip Time, a feature in TLS 1.3 and QUIC that allows a client to send application data in its very first message to a server when resuming a connection, thus eliminating an entire round trip of latency from the connection establishment process.

**BC** Boundary Clock, a device that acts as both a slave to an upstream master clock, synchronizing its own time, and then as a master to downstream slave clocks, distributing that synchronized time to other network segments.

**BMCA** Best Master Clock Algorithm, a distributed selection process where clocks exchange "Announce" messages containing their properties to determine the highest-quality clock to serve as the grandmaster for the entire network.

**BPF** Berkeley Packet Filter, in-kernel virtual machine that allows sandboxed bytecode to be executed at various hook points.

**CDF** Cumulative Distribution Function, describes the probability that some random variable $X$ will have a value less than or equal to $x$.

**cBPF** classic Berkeley Packet Filter, a minimalistic, in-kernel bytecode language and virtual machine primarily used for network packet inspection and filtering.

**CI** Continuous Integration, an automated validation of code merges through builds and tests.

**CID** Connection ID, a chosen identifier used by endpoints to identify a QUIC connection, enabling it to persist and be routed correctly even if the underlying IP address or port changes

**CPU** Central Processing Unit, the "brain" of the computer, where most programs are run.

**DDoS** Distributed Denial of Service, a malicious attempt to disrupt the normal traffic of a targeted service by flooding it with internet traffic from multiple sources.

**DMA** Direct Memory Access, a feature that allows hardware components to directly access system memory for reading and writing data without requiring the CPU's intervention.

**DPDK** Data Plane Development Kit, a collection of open-source libraries and drivers that enable high-performance, kernel-bypass packet processing in userspace.

**eBPF** extended Berkeley Packet Filter, an extended, in-kernel bytecode language and virtual machine, which expands on cBPF's capabilities to allow for programmable, event-driven execution across the kernel's subsystems.

**GCC** GNU Compiler Collection, a compiler system for various programming languages like C and C++.

**GMC** Grandmaster Clock, an authoritative time source for a Precision Time Protocol (PTP) network.

**GPS** Global Positioning System, a satellite-based navigation system that provides users with accurate positioning, navigation, and timing information anywhere on Earth.

**gPTP** Generalized Precision Time Protocol, an adaptation of the Precision Time Protocol specifically designed to provide highly accurate time synchronization for time-sensitive applications within Ethernet networks.

**GRO** Generic Receive Offload, a software technique that aggregates multiple incoming small packets belonging to the same flow into a single, larger packet within the NIC driver, which reduces the number of packets the CPU has to process and improves receive throughput.

**GSO** Generic Segmentation Offload, a software technique that splits chunks of data belonging to the same flow into smaller MTU-sized packets, typically at the driver level, which improves sending throughput.

**HoL** Head of Line

**HTTP/3** Hypertext Transfer Protocol 3, the latest version of the Hypertext Transfer Protocol, which leverages QUIC over UDP to eliminate head-of-line blocking.

**IEEE** Institute of Electrical and Electronics Engineers, an organization that develops and publishes standards for electrical and computing technologies.

**JIT** Just in Time, a method where code is translated from an intermediate representation into native machine code during runtime, immediately before execution, to improve performance.

**kTLS** Kernel Transport Layer Security, a Linux kernel feature that offloads the encryption and decryption of TLS records from userspace applications to the kernel, aiming to improve performance by reducing data copies and enabling hardware acceleration.

**MITM** Man-in-the-Middle, when an adversary positions itself between two communicating parties to intercept and manipulate their data, typically without their knowledge.

**NIC** Network Interface Card, a hardware component that connects a computer to a computer network, allowing it to send and receive data.

**OC** Ordinary Clock, a device with a single PTP port that can either be a source of time (master) or a receiver of time (slave).

**PAL** Platform Abstraction Layer, a software layer in MsQuic, separated from the core QUIC logic, which holds the abstractions for TLS, UDP, and the OS.

**PHC** PTP Hardware Clock, a highly accurate clock integrated directly into network interface cards to provide the precise timestamping of PTP packets. These clocks are necessary for the sub-microsecond resolution present in PTP.

**P2P** Peer to Peer

**PTP** Precision Time Protocol, an IEEE standard that enables nanosecond-level clock synchronization across computer networks.

**QIR** QUIC Interop Runner, an automated test framework that runs various test cases between different QUIC client and server implementations to assess and ensure their interoperability.

**RISC** Reduced Instruction Set Computer, an instruction set architecture that prioritizes simpler, fixed-length instructions for faster execution and simpler processor design.

**RPS** Requests per Second, a metric showing the number of requests a server can handle within a second.

**RSS** Receive Side Scaling, a network driver feature that distributes the processing of incoming network packets across multiple CPU cores to improve network throughput and reduce CPU bottlenecks.

**TAI** International Atomic Time, an extremely precise atomic timescale based on the weighted average of the readings of atomic clocks worldwide.

**TC** Transparent Clock, a device that forwards PTP messages, and compensates for internal latency by adding its own processing time to the packet's correction field.

**TCP** Transmission Control Protocol, an internet protocol that provides reliable, ordered, and error-checked delivery of data streams between applications over an IP network.

**TLS** Transport Layer Security, a cryptographic protocol that provides end-to-end security for data transmitted over a network, ensuring privacy, integrity, and authenticity between communicating applications. It is most notably used to secure web traffic via HTTPS.

**TSN** Time Sensitive Networking, a set of standards that provide deterministic messaging and guaranteed delivery for time-sensitive data over Ethernet networks.

**UDP** User Datagram Protocol, a lightweight, connectionless protocol that QUIC uses as its layer 4 transport.

**UTC** Coordinated Universal Time, the world's primary time standard, based on International Atomic Time (TAI) but adjusted by leap seconds to account for Earth's irregular rotation.

**XDP** EXpress Data Path, a programmable hook within the Linux kernel allowing for early packet processing before the network stack.

**XSK** XDP Socket, a Linux kernel socket and part of the AF_XDP address family, which allows user-space applications to directly access raw network packets.

# Bibliography


[1] W. Eddy, *Transmission Control Protocol (TCP)*, RFC 9293, Aug. 2022. DOI: 10.17487/RFC9293. [Online]. Available: https://www.rfc-editor.org/info/rfc9293.

[2] E. Rescorla, *The Transport Layer Security (TLS) Protocol Version 1.3*, RFC 8446, Aug. 2018. DOI: 10.17487/RFC8446. [Online]. Available: https://www.rfc-editor.org/info/rfc8446.

[3] J. Iyengar and M. Thomson, *QUIC: A UDP-Based Multiplexed and Secure Transport*, RFC 9000, May 2021. DOI: 10.17487/RFC9000. [Online]. Available: https://www.rfc-editor.org/info/rfc9000.

[4] M. Bishop, *HTTP/3*, RFC 9114, Jun. 2022. DOI: 10.17487/RFC9114. [Online]. Available: https://www.rfc-editor.org/info/rfc9114.

[5] Cloudflare, *Cloudflare radar year in review 2024*, Accessed: 2024-12-14, 2024. [Online]. Available: https://radar.cloudflare.com/year-in-review/2024#http-versions.

[6] B. Jaeger, J. Zirngibl, M. Kempf, K. Ploch, and G. Carle, "Quic on the highway: Evaluating performance on high-rate links", in *2023 IFIP Networking Conference (IFIP Networking)*, 2023, pp. 1–9. DOI: 10.23919/IFIPNetworking57963.2023.10186365.

[7] D. S. Miller, *Merge branch 'xdp': Add driver bpf hook for early packet drop and forwarding*, https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=22b3548861fb21ad79e0d3afeee123b0eb3912cc, Commit hash: 22b3548861fb21ad79e0d3afeee123b0eb3912cc, 2016.

[8] The Linux Foundation, *Data Plane Development Kit (DPDK)*, https://www.dpdk.org/, The Open Source Data Plane Development Kit Accelerating Network Performance., 2024.

[9] X. Long, *Quic in linux kernel*, https://github.com/lxin/quic, Accessed: 2025-01-13, 2025.

[10] J. Späth, “Quic performance improvements using socket optimizations and kernel bypass”, Master’s thesis, Technical University of Munich, Garching, Bavaria, 2023.

[11] T. C. Blog. “A quic update on google’s experimental transport protocol”. (2015), [Online]. Available: https://blog.chromium.org/2015/04/a-quic-update-on-googles-experimental.html (visited on 07/01/2025).

[12] T. C. Blog. “Making chrome quicer”. (2024), [Online]. Available: https://blog.chromium.org/2024/12/making-chrome-quicer.html (visited on 07/01/2025).

[13] M. Thomson, *Greasing the QUIC Bit*, RFC 9287, Aug. 2022. DOI: 10.17487/RFC9287. [Online]. Available: https://www.rfc-editor.org/info/rfc9287.

[14] A. Starovoitov, *bpf: add generic bpf(2) syscall*, Git commit b4fc1a460f3017e958e6a8ea560ea0afd91bf
to Linux kernel, Accessed: June 11, 2025, Sep. 2014. [Online]. Available: https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=b4fc1a460f3017e958e6a8ea560ea0afd91bf6fe.

[15] Isovalent. “Af_xdp”, Isovalent. (2025), [Online]. Available: https://docs.ebpf.io/linux/concepts/af_xdp/ (visited on 07/01/2025).

[16] J. Kicinski and N. Viljoen, “XDP Hardware Offload: Current Work, Debugging and Edge Cases”, in *Proceedings of Netdev 2.2*, Talk presented at Netdev 2.2, November 8-10, 2017, Seoul, South Korea, 2017. [Online]. Available: https://netdevconf.info/2.2/papers/viljoen-xdpoffload-talk.pdf.

[17] *IEEE Standard for a Precision Clock Synchronization Protocol for Networked Measurement and Control Systems*, (Revision of IEEE Std 1588-2008), IEEE, 2020.

[18] O. Obleukhov and A. Byagowi. “How precision time protocol is being deployed at meta”. Meta Engineering Blog, Meta Engineering. (Nov. 2022), [Online]. Available: https://engineering.fb.com/2022/11/21/production-engineering/precision-time-protocol-at-meta/ (visited on 07/02/2025).

[19] M. D. Johas Teener and G. M. Garner, “Overview and timing performance of ieee 802.1as”, in *2008 IEEE International Symposium on Precision Clock Synchronization for Measurement, Control and Communication*, 2008, pp. 49–53. DOI: 10.1109/ISPCS.2008.4659212.

[20] R. Cochran. “Linuxptp: Precision time protocol (ptp) for linux”, The LinuxPTP Project. (), [Online]. Available: https://github.com/richardcochran/linuxptp (visited on 07/03/2025).

[21] K. Rösel, M. Helm, J. Zirngibl, and H. Stubbe, “Current developments of ieee 1588 (precision time protocol)”, in *Seminar IITM WS 20/21, Network Architectures and Services*, doi: 10.2313/NET-2021-05-1_04, Technical University of Munich, 2021. [Online]. Available: https://www.net.in.tum.de/fileadmin/TUM/NET/NET-2021-05-1/NET-2021-05-1_04.pdf.

[22] R. Molenaar. "Introduction to precision time protocol (ptp)", NetworkLessons.com. (2024), [Online]. Available: https://networklessons.com/ip-services/introduction-to-precision-time-protocol-ptp (visited on 07/03/2025).

[23] Cisco, *Configuring precision time protocol (ptp)*, Catalyst 9300 Series Switches Software Release 17.1 Configuration Guide, Cisco, 2023. (visited on 07/07/2025).

[24] Mobatime. "Precision time protocol (ptp)", Mobatime. (), [Online]. Available: https://www.mobatime.com/technology/ptp-precision-time-protocol/ (visited on 07/03/2025).

[25] M. Kempf, B. Jaeger, J. Zirngibl, K. Ploch, and G. Carle, "QUIC on the Fast Lane: Extending Performance Evaluations on High-rate Links", *Computer Communications*, vol. 223, pp. 90–100, Jul. 2024, ISSN: 0140-3664. DOI: 10.1016/j.comcom.2024.04.038. [Online]. Available: https://doi.org/10.1016/j.comcom.2024.04.038.

[26] Microsoft, *Architecture*, Accessed on 6 July 2025, 2025. [Online]. Available: https://github.com/microsoft/msquic/blob/main/docs/Architecture.md.

[27] Microsoft, *Diagnostics*, Accessed on 6 July 2025, 2025. [Online]. Available: https://github.com/microsoft/msquic/blob/main/docs/Diagnostics.md.

[28] G. Martínez, J. A. Hernández, P. Reviriego, and P. Reinheimer, "Round trip time (rtt) delay in the internet: Analysis and trends", *IEEE Network*, vol. 38, no. 2, pp. 280–285, 2024. DOI: 10.1109/MNET004.2300008.

[29] M. Kempf, S. Tietz, B. Jaeger, J. Späth, G. Carle, and J. Zirngibl, "Quic steps: Evaluating pacing strategies in quic implementations", *Proceedings of the ACM on Networking*, vol. 3, no. CoNEXT2, 1–14, Jun. 2025, ISSN: 2834-5509. DOI: 10.1145/3730985. [Online]. Available: http://dx.doi.org/10.1145/3730985.

[30] N. Tyunyayev, "Improving the performance of picoquic by bypassing the linux kernel with dpdk", Master's thesis, École polytechnique de Louvain, Ottignies-Louvain-la-Neuve, Belgium, 2021.

[31] Private Octopus Consulting, *Picoquic: Minimal Implementation of the QUIC Protocol*, https://github.com/private-octopus/picoquic, Minimalist implementation of the QUIC protocol., 2025.

[32] T. Huang and S. Zhao, "Accelerating quic with af_xdp", in *Algorithms and Architectures for Parallel Processing: 23rd International Conference, ICA3PP 2023, Tianjin, China, October 20–22, 2023, Proceedings, Part III*, Tianjin, China: Springer-Verlag, 2024, 89–100, ISBN: 978-981-97-0797-3. DOI: 10.1007/978-981-97-0798-0_6. [Online]. Available: https://doi.org/10.1007/978-981-97-0798-0_6.

[33] M. Karlsson and B. Töpel, "The path to dpdk speedsfor af_xdp", in *Linux Plumbers Conference 2018*, 2018. [Online]. Available: http://oldvger.kernel.

org/lpc_net2018_talks/lpc18_paper_af_xdp_perf-v2.pdf (visited on 07/11/2025).

[34] L. Bianconi, E. Chaudron, J. D. Brouer, and T. Høiland-Jørgensen, *Bringing tso/-gro and jumbo frames to xdp*, Linux Plumbers Conference, Contribution to Linux Plumbers Conference 2021, 2021. [Online]. Available: https://lpc.events/event/11/contributions/939/attachments/771/1551/xdp-multi-buff.pdf (visited on 07/11/2025).

[35] Isovalent. "Program type bpf_prog_type_xdp", Isovalent. (2025), [Online]. Available: https://docs.ebpf.io/linux/program-type/BPF_PROG_TYPE_XDP/#driver-support (visited on 07/01/2025).

[36] B. Jaeger, J. Zirngibl, M. Kempf, K. Ploch, and G. Carle, "Quic on the highway: Evaluating performance on high-rate links", Jun. 2023, pp. 1–9. DOI: 10.23919/IFIPNetworking57963.2023.10186365.

[37] X. Yang, L. Eggert, J. Ott, S. Uhlig, Z. Sun, and G. Antichi, "Making quic quicker with nic offload", in *Proceedings of the Workshop on the Evolution, Performance, and Interoperability of QUIC*, ser. EPIQ '20, Virtual Event, USA: Association for Computing Machinery, 2020, 21–27, ISBN: 9781450380478. DOI: 10.1145/3405796.3405827. [Online]. Available: https://doi.org/10.1145/3405796.3405827.